

HOCHSCHULE AALEN
TECHNIK, WIRTSCHAFT UND GESUNDHEIT

Fakultät Elektronik und Informatik
Masterstudiengang Informatik und Security (M. Sc.)

Projektarbeit

Navigation in prozedural generierten 2D-Welten mit Reinforcement Learning

Generalisierung eines rekurrenten Agenten auf
unbekannte Welten am Beispiel *Stoneforge*

Vorgelegt von: Florian Merlau
Laurin Rößler

Betreuer: Prof. Dr. Carsten Lecon

Abgabedatum: 25. Juli 2026

Eidesstattliche Erklärung

Wir versichern hiermit, dass wir die vorliegende Projektarbeit mit dem Thema „Navigation in prozedural generierten 2D-Welten mit Reinforcement Learning“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen oder anderen Quellen entnommen sind, sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

Eingesetzte Hilfsmittel sind im Anhang [9](#) dokumentiert; der vollständige Quellcode sowie das lückenlos geführte Experiment-Changelog liegen dem Projekt-Repository bei.

Ort, Datum

Florian Merlau

Ort, Datum

Laurin Rößler

Kurzfassung

Diese Projektarbeit untersucht, ob ein Reinforcement-Learning-Agent in prozedural generierten, nur teilweise einsehbaren 2D-Welten eine Navigationsstrategie lernt, die auf unbekannte Welten übertragbar ist. Als Versuchsplattform dient das eigens entwickelte Spiel *Stoneforge* (C++-Kern, Python-Anbindung). Der Agent sieht nur einen lokalen 15×15 -Ausschnitt und muss den Ausgang finden. Trainiert wird RecurrentPPO (PPO mit LSTM-Gedächtnis) mit einem vierstufigen Curriculum; bewertet wird die Erfolgsquote auf 50 trainingsfremden Welten, gemittelt über sieben unabhängige Trainingsläufe.

Der Agent erreicht $65,7\% \pm 12,4$ auf dem Testset und $66,9\% \pm 12,8$ auf dem Holdout und erfüllt damit das Holdout-, nicht aber das Testset-Kriterium. Über dieses Einzelergebnis hinaus liefert die Arbeit zwei methodische Beiträge. Erstens einen kalibrierten Bezugspunkt: Eine ungelernte Kompass-Heuristik erreicht unter identischem Protokoll 92 % und ist bei vergleichbarer Erfolgsquote dreimal wegeffizienter als der trainierte Agent. Die Erfolgsquote ist in dieser Parametrisierung folglich nicht trennscharf, denn die Umgebung erfordert das Gedächtnis nicht, dessen Nutzen sie messen soll. Zweitens die Quantifizierung der Streuung über Trainingsläufe: Die Aufstockung von drei auf sieben Läufe senkte das Ergebnis um rund acht Prozentpunkte; wenige Läufe führen hier systematisch zu optimistischen Aussagen. Aus beiden Befunden leitet die Arbeit ein korrigiertes Messregime für Folgearbeiten ab und stellt mit der Referenzpolitik das Instrument bereit, mit dem sich die Aufgabenschwierigkeit vorab kalibrieren lässt.

Schlagwörter: Reinforcement Learning, POMDP, Generalisierung, prozedurale Generierung, RecurrentPPO, Curriculum Learning, Baseline-Vergleich

Inhaltsverzeichnis

Eidesstattliche Erklärung	ii
Kurzfassung	iii
Abbildungsverzeichnis	vi
Tabellenverzeichnis	vii
Abkürzungs- und Symbolverzeichnis	viii
1 Einleitung	1
1.1 Motivation	1
1.2 Die Aufgabe in Kürze	1
1.3 Problemstellung und Forschungsfrage	1
1.4 Zielsetzung und Erfolgskriterien	2
1.5 Aufbau der Arbeit	2
2 Grundlagen	3
2.1 Reinforcement Learning und der Markov-Entscheidungsprozess	3
2.2 Partielle Beobachtbarkeit (POMDP)	3
2.3 Wertbasierte Verfahren: Q-Learning und DQN	4
2.4 Policy-Gradient- und Actor-Critic-Verfahren	5
2.5 Proximal Policy Optimization (PPO)	5
2.6 Rekurrente Politiken (LSTM)	6
2.7 Reward Shaping (PBRs)	6
2.8 Curriculum Learning	7
2.9 Prozedurale Generierung und Generalisierung	7
3 Einordnung in verwandte Arbeiten	7
4 Die Umgebung Stoneforge	8
4.1 POMDP-Charakter	8
4.2 Beobachtung, Aktionen, Episodenende	9
4.3 Reward-Design	9
4.4 Garantierte Lösbarkeit	10
5 Methodik	10
5.1 Algorithmus	10
5.2 Curriculum	11
5.3 Iterative Modellverbesserung	11
5.4 Evaluationsprotokoll	12
5.5 Herleitung der Erfolgsschwellen	12
5.6 Versuchsübersicht: Setup je Iteration	13
6 Technische Umsetzung und Implementierung	14
6.1 Technologie-Stack	14
6.2 Umgebung und Binding	14
6.3 Trainings-Pipeline	14
6.4 Monitoring und Werkzeuge	15
7 Evaluation	15
7.1 Zentrale Gegenüberstellung: Gedächtnis statt lokalem Orakel	15

7.2	Endergebnis (v12, $n = 7$)	16
7.3	Ungelernte Referenzpolitiken und die Grenzen der Erfolgsquote	18
7.4	Der Det/Stoch-Gap: Weglängen-Analyse und POMDP-Einordnung	20
7.5	Versuche zur Schließung des Det/Stoch-Gaps	21
8	Fazit und Ausblick	22
8.1	Ergebnis	22
8.2	Beiträge	22
8.3	Einschränkungen	23
8.4	Ausblick	23
9	Anhang: Reproduzierbarkeit	24
10	Anhang: Entwicklungsverlauf von der Baseline zum LSTM-Agenten	25
10.1	Etappe 1 (Mai 2026): Baselines mit BFS-Orakel	25
10.2	Etappe 2 (Juni 2026): Kernbeitrag, Navigation ohne Orakel	26
10.3	Etappe 3 (13.–25.06.): Fehlkonfigurierte Hyperparameter und Root-Cause-Analyse	27
10.4	Etappe 4 (06.07.2026): Umgebungsrevision v11 und Performance	27
10.5	Etappe 5 (07.07.2026): Erster v11-Gesamtlauf, Monitoring-Redesign und Seed- Replay-Analyse	27
10.6	Etappe 6 (07.07.2026, nachmittags/abends): Systematische Fehlersuche vom Eval-Artefakt zur Batch-Größen-Korrektur	29
10.7	Umgebungsrevision v11 (06.07.2026)	30
10.8	Performance-Analyse: Training beschleunigen	32
10.8.1	CPU schlägt Apple-GPU (MPS)	32
10.8.2	Batch-Größe: vermeintliche Validierung (06.07.), am Folgetag widerlegt	32
10.9	Zeitleiste und Lessons Learned	33
	Literaturverzeichnis	36

Abbildungsverzeichnis

1	Agent-Umwelt-Interaktion im MDP	4
2	MDP gegenüber POMDP	4
3	Partielle Beobachtbarkeit am Beispiel Stoneforge	5
4	Wirkung des PPO-Clippings	6
5	Rekurrente Politik über die Zeit	7
6	Benutzeroberfläche des spielbaren C++-Clients	9
7	Lernkurve des historischen Referenzlaufs	16
8	Trainingsverläufe der sieben v12-Läufe	17
9	Erfolgsquote nach Startdistanz zum Exit	21
10	Entwicklung über die Modellgenerationen	26
11	Lernkurve des ersten v11-Gesamtlaufs	28
12	Live-Monitoring-Dashboard	28
13	Eval-Cap-Artefakt	29
14	Batch-Größen-Forensik	31
15	Wirkung der BFS-basierten Exit-Platzierung	32
16	Training-FPS nach Device und Batch-Größe	33
17	Validierungslauf batch_size=64	34

Tabellenverzeichnis

2	Aufbau der Observation (Env v11)	9
3	Reward-Komponenten (Env v11)	10
4	Curriculum in vier Phasen	11
5	Gemeinsame Basiskonfiguration aller Iterationen	13
6	Systematische Iterationen zur Schließung des Det/Stoch-Gaps	13
7	Technologie-Stack.	14
8	Gegenüberstellung: Gedächtnis gegenüber lokalem BFS-Orakel	15
9	Endergebnis der v12-Läufe über sieben Seeds	16
10	Ungelernte Referenzpolitiken gegenüber dem trainierten Agenten	19
11	Erfolgsquote als Funktion des Episodenbudgets	19
12	Temperatur-Sweep der Evaluation	22
13	Systemumgebung	24
14	Root-Cause-Analyse der fehlgeschlagenen Läufe v7–v9	27
15	Zeitleiste der RL-Entwicklung	35

Abkürzungs- und Symbolverzeichnis

Kürzel	Bedeutung
A2C	Advantage Actor-Critic, synchrone Variante von A3C
approx_kl	Maß für die Stärke einer Politikänderung pro Update
BFS	<i>Breadth-First Search</i> , Breitensuche; liefert den kürzesten begehbaren Weg
BPTT	<i>Backpropagation Through Time</i> , Lernverfahren rekurrenter Netze
CNN	<i>Convolutional Neural Network</i> , faltendes neuronales Netz
DQN	<i>Deep Q-Network</i> , wertbasiertes RL-Verfahren
DRQN	<i>Deep Recurrent Q-Network</i> , DQN mit Gedächtnis
EV	<i>Explained Variance</i> , Gütemaß der gelernten Wertfunktion
GAE	<i>Generalized Advantage Estimation</i> , Schätzung des Vorteils
HARKing	<i>Hypothesizing After the Results are Known</i>
LSTM	<i>Long Short-Term Memory</i> , rekurrente Netzarchitektur (Gedächtnis)
MDP	<i>Markov Decision Process</i> , Markow-Entscheidungsprozess
MLP	<i>Multilayer Perceptron</i> , vorwärtsgerichtetes Netz ohne Gedächtnis
MPS	<i>Metal Performance Shaders</i> , GPU-Backend von Apple
PBRs	<i>Potential-Based Reward Shaping</i> , politikerhaltende Belohnungsformung
PLR	<i>Prioritized Level Replay</i> , priorisierte Level-Wiederholung
POMDP	<i>Partially Observable MDP</i> , teilweise beobachtbarer MDP
PPO	<i>Proximal Policy Optimization</i> , das hier verwendete RL-Hauptverfahren
RL	<i>Reinforcement Learning</i> , bestärkendes Lernen
SB3	<i>Stable-Baselines3</i> , verwendete RL-Bibliothek
SR	<i>Success Rate</i> , Erfolgsquote: Anteil gelöster Welten
γ	Diskontfaktor: Gewicht künftiger Belohnungen (hier 0,999)
ε	Zufallsanteil der Vergleichspolitik (Abschnitt 7.3)
η	Pfadeffizienz: kürzester Weg geteilt durch tatsächliche Schritte
$\pi(a \mid s)$	Politik: Wahrscheinlichkeit, im Zustand s die Aktion a zu wählen
$\Phi(s)$	Potentialfunktion des Reward Shapings
σ	Standardabweichung über Trainingsläufe

1 Einleitung

1.1 Motivation

Ein Computerprogramm, das ein Videospiel-Level fehlerfrei durchspielt, hat zunächst nur eines bewiesen: dass es *dieses* Level beherrscht. Interessant wird es erst, wenn es ein Level meistert, das es nie zuvor gesehen hat. Genau diese Unterscheidung, zwischen Auswendiglernen und tatsächlichem Lernen einer Strategie, ist die zentrale offene Frage beim praktischen Einsatz von *Reinforcement Learning* (RL, bestärkendes Lernen): einem Verfahren, bei dem ein Programm (der *Agent*) nicht durch vorgegebene Regeln, sondern durch Versuch, Irrtum und Belohnung lernt.

Die Übertragbarkeit auf Unbekanntes heißt in der Fachsprache *Generalisierung*. Sie zu prüfen ist schwierig, weil ein Agent, der immer dieselbe Umgebung sieht, deren Eigenheiten mitlernt, ohne dass es auffällt. Ein etabliertes Gegenmittel ist die *prozedurale Generierung*: Die Umgebung wird für jeden Durchlauf neu per Zufallszahl erzeugt, sodass Auswendiglernen nichts nützt [1, 2]. Auf diesem Prinzip beruht die vorliegende Arbeit.

1.2 Die Aufgabe in Kürze

Für diese Arbeit wurde ein eigenes 2D-Spiel entwickelt: *Stoneforge*. Die Spielwelt besteht aus einem Raster aus Feldern (Wiese, Wald, Fels, Wasser), das aus einer Zufallszahl, dem sogenannten *Seed*, erzeugt wird. Derselbe Seed erzeugt immer dieselbe Welt, ein anderer Seed eine völlig andere. Irgendwo in dieser Welt liegt ein Ausgang (*Exit*); der Agent startet in der Mitte und soll ihn erreichen. Er darf pro Zug ein Feld nach oben, unten, links oder rechts gehen.

Die Schwierigkeit liegt darin, dass der Agent die Welt nicht überblickt. Er sieht nur einen Ausschnitt von 15×15 Feldern um sich herum; alles darüber hinaus ist für ihn schwarz. Zusätzlich kennt er die *Richtung*, in der der Ausgang liegt, so wie ein Kompass die Himmelsrichtung anzeigt, aber nicht den *Weg* dorthin. Steht eine lange Felswand zwischen ihm und dem Ziel, muss er selbst herausfinden, ob er sie nördlich oder südlich umgehen soll, und sich dabei merken, wo er schon war. Abbildung 3 auf Seite 5 stellt diesen Unterschied zwischen Weltzustand und Agentensicht gegenüber.

Gelernt wird durch Belohnung: Erreicht der Agent den Ausgang, erhält er einen großen positiven Wert; jeder Schritt kostet ein wenig. Über Millionen von Versuchen passt er sein Verhalten so an, dass die Summe der Belohnungen möglichst groß wird. Gemessen wird am Ende die *Erfolgsquote* (*Success Rate*): der Anteil von 50 vorher nie gesehenen Welten, in denen er den Ausgang findet.

1.3 Problemstellung und Forschungsfrage

Aus dem beschriebenen Aufbau folgt die eigentliche Schwierigkeit: Weil der Agent nur einen Ausschnitt sieht, sind zwei völlig verschiedene Stellen der Welt für ihn unter Umständen nicht unterscheidbar. Die naheliegende Abhilfe wäre, ihm den richtigen Weg vorzurechnen und als zusätzliche Eingabe mitzugeben, ein „Orakel“. Dann löst er die Aufgabe zwar, hat aber nichts gelernt, was ohne dieses Orakel funktioniert.

Daraus ergibt sich die Forschungsfrage dieser Arbeit:

Kann ein Reinforcement-Learning-Agent in prozedural generierten, nur teilweise einsehbaren 2D-Welten eine Navigationsstrategie lernen, die auch auf unbekannten Welten funktioniert, ohne dass ihm der Weg zum Ziel vorberechnet wird?

Die Frage ist bewusst offen nach der Architektur gestellt. Wie ein Agent das leisten könnte, ist selbst Teil des Untersuchungsgegenstands: Dass dafür ein internes Gedächtnis erforderlich

ist, war zu Projektbeginn eine Vermutung und ergab sich erst im Verlauf aus dem Scheitern gedächtnisloser Varianten (Anhang 10).

1.4 Zielsetzung und Erfolgskriterien

Die Erfolgskriterien, gegen die in dieser Arbeit berichtet wird, lauten:

- **Testset A** (Seeds 7000–7049): $\geq 70\%$ Erfolgsquote,
- **Holdout B** (Seeds 8000–8049): $\geq 60\%$ Erfolgsquote,
- pro Konfiguration mindestens 3 Trainingsläufe (Mittelwert $\pm$ Standardabweichung).

Beide Schwellen wurden vor den Auswertungsläufen festgelegt; ihre Herleitung aus den Eigenschaften der Aufgabe folgt in Abschnitt 5.5.

1.5 Aufbau der Arbeit

Die Arbeit folgt dem üblichen Ablauf empirischer Untersuchungen: erst die Begriffe, dann der Aufbau des Versuchs, dann die Messungen, zuletzt deren Bewertung.

- **Kapitel 2 (Grundlagen)** erklärt die verwendeten Verfahren von Grund auf: was ein Markov-Entscheidungsprozess ist, warum ein eingeschränktes Sichtfeld die Aufgabe grundlegend verändert, wie die eingesetzten Lernverfahren arbeiten und was Gedächtnis, Belohnungsformung und Curriculum bedeuten. Wer mit RL vertraut ist, kann es überspringen.
- **Kapitel 3 (Verwandte Arbeiten)** ordnet die Aufgabe in bestehende Forschungsumgebungen ein und begründet, warum ein Zahlenvergleich mit ihnen nicht seriös möglich ist.
- **Kapitel 4 (Umgebung)** beschreibt *Stoneforge* im Detail: was der Agent genau sieht, welche Aktionen er hat, wie er belohnt wird und warum jede erzeugte Welt nachweislich lösbar ist.
- **Kapitel 5 (Methodik)** legt das Vorgehen offen: Lernverfahren und Einstellungen, das stufenweise Training, die strikte Trennung der Weltmengen für Training, Auswahl und Messung sowie die Begründung der Erfolgsschwellen.
- **Kapitel 6 (Technische Umsetzung)** dokumentiert die Implementierung: C++-Kern, Python-Anbindung, Trainingspipeline und Werkzeuge.
- **Kapitel 7 (Evaluation)** berichtet die Messergebnisse: die Gegenüberstellung mit und ohne Orakel, das Endergebnis über sieben Läufe, den Vergleich gegen ungelernete Referenzpolitiken und die Analyse des Abstands zwischen den beiden Auswertungsarten.
- **Kapitel 8 (Fazit und Ausblick)** bewertet die Ergebnisse gegen die Forschungsfrage, benennt die Grenzen der Arbeit und leitet daraus die nächsten Schritte ab.

Zur Versionsbezeichnung. Im Folgenden tauchen wiederholt Kürzel wie *v11* oder *v12* auf. Sie bezeichnen Entwicklungsstände des Projekts und sind nötig, weil Ergebnisse verschiedener Stände *nicht* vergleichbar sind: Ändert sich die Weltgenerierung, erzeugt derselbe Seed eine andere Welt. Zwei Stände sind für das Verständnis ausreichend:

- **v11** bezeichnet die am 06.07.2026 überarbeitete Umgebung. Sie ist der Stand, auf dem alle berichteten Endergebnisse beruhen. Wesentliche Änderungen: Die Exit-Entfernung wird seither über den tatsächlichen Laufweg statt über die Luftlinie festgelegt, wirkungslose Beobachtungsmerkmale wurden entfernt und mehrere Strafterme entschärft (Details in Abschnitt 10.7).
- **v12** bezeichnet die darauf aufbauende *Trainings*-Konfiguration vom 08.07.2026, mit der die sieben finalen Läufe erzeugt wurden. Die Umgebung ist dabei unverändert v11; v12 unterscheidet sich nur in den Trainingseinstellungen (Tabelle 5).

Ältere Stände (v2 bis v10) kommen ausschließlich im Entwicklungsverlauf (Anhang 10) vor. Wo im Hauptteil ältere Zahlen zitiert werden, ist der zugehörige Stand jeweils angegeben.

Zwei Anhänge ergänzen den Hauptteil: Anhang 9 führt alle Angaben zusammen, die zur Reproduktion der Ergebnisse nötig sind; Anhang 10 dokumentiert den chronologischen Entwicklungsverlauf einschließlich der gescheiterten Versuche und ihrer Ursachenanalysen. Mehrere zentrale Erkenntnisse der Arbeit, darunter die Wirkung der Batch-Größe und das Eval-Cap-Artefakt, ergaben sich erst aus dem Vergleich fehlgeschlagener mit funktionierenden Läufen und sind deshalb dort nachzulesen.

2 Grundlagen

Zwei Entscheidungen dieser Arbeit lassen sich erst vor dem Hintergrund der folgenden Begriffe begründen: die Wahl einer rekurrenten statt einer reaktiven Politik und die Wahl der stochastischen Auswertung als Primärmetrik. Für eine vollständige Einführung in das Gebiet sei auf Sutton und Barto [3] verwiesen.

2.1 Reinforcement Learning und der Markov-Entscheidungsprozess

Reinforcement Learning (RL) beschreibt das Lernen durch Interaktion: Ein Agent wählt in einem Zustand eine Aktion, erhält eine Belohnung (Reward) und geht in einen Folgezustand über [3]. Formal wird dies als Markov-Entscheidungsprozess (MDP) modelliert, ein Tupel (S, A, P, R, γ) mit Zustandsmenge S , Aktionsmenge A , Übergangswahrscheinlichkeit $P(s' | s, a)$, Rewardfunktion $R(s, a)$ und Diskontfaktor $\gamma \in [0, 1)$. Ziel ist eine Politik $\pi(a | s)$, die den erwarteten diskontierten Return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ maximiert. Die Zustandswertfunktion $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ gibt den erwarteten Return ab einem Zustand an. Der Diskontfaktor γ gewichtet spätere Rewards geringer; im Projekt wird $\gamma = 0,999$ verwendet.

Abbildung 1 zeigt diesen Kreislauf. Er ist die Grundfigur des gesamten Verfahrens: Alles, was in dieser Arbeit trainiert wird, ist eine Variante dieser Schleife, in der die Umgebung *Stoneforge* heißt, die Aktionen die vier Bewegungsrichtungen sind und der Reward aus Abschnitt 4 stammt.

2.2 Partielle Beobachtbarkeit (POMDP)

Sieht der Agent nicht den vollständigen Zustand, sondern nur eine Beobachtung, liegt ein *Partially Observable* MDP (POMDP) vor [4]. Die Markov-Eigenschaft gilt dann für die Beobachtung nicht mehr: Dieselbe lokale Beobachtung kann zu verschiedenen globalen Zuständen gehören. Eine reaktive Politik, die nur die aktuelle Beobachtung nutzt, ist deshalb im Allgemeinen nicht optimal. Abhilfe schafft ein internes Gedächtnis, das vergangene Beobachtungen zu einem *Belief-State* zusammenfasst [5]. Abbildung 2 stellt beide Fälle gegenüber: Im MDP greift die Politik direkt auf den Zustand zu, im POMDP liegt eine Beobachtungsfunktion dazwischen, und die Politik muss aus der Vorgeschichte erst rekonstruieren, wo sie sich vermutlich befindet.

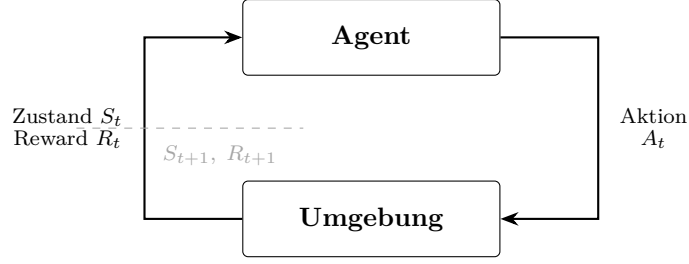
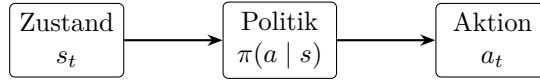


Abbildung 1: Die Agent-Umwelt-Interaktion im Markov-Entscheidungsprozess; die gestrichelte Linie markiert den Übergang zum nächsten Zeitschritt. Eigene Darstellung in Anlehnung an Sutton und Barto [3, Abb. 3.1].

MDP: vollständig beobachtbar



POMDP: nur Ausschnitt sichtbar

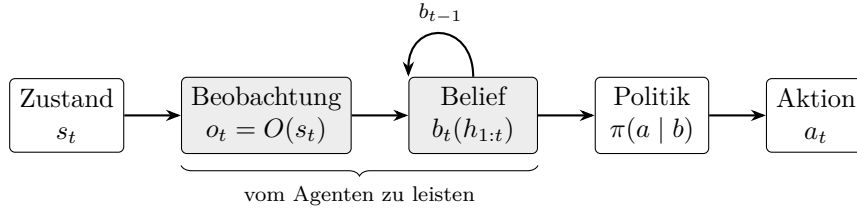


Abbildung 2: Vollständige (oben) gegenüber partieller Beobachtbarkeit (unten). Grau hinterlegt sind die beiden Schritte, die im POMDP zusätzlich anfallen: die Beobachtung $o_t = O(s_t)$ und die Bildung eines *Beliefs* aus ihr und der Vorgeschichte. Eigene Darstellung; Formalismus nach Kaelbling et al. [4].

Eine zweite Konsequenz ist für diese Arbeit zentral: Unter partieller Beobachtbarkeit kann eine *stochastische* gedächtnislose Politik jede deterministische gedächtnislose erheblich übertreffen [6]. Zu beachten ist die Reichweite dieser Aussage: Sie gilt für *gedächtnislose* Politiken. Ist der Belief-State hinreichend genau rekonstruiert, existiert wieder eine optimale deterministische Politik. Ein Leistungsabstand zwischen Sampling- und Argmax-Auswertung einer *rekurrenten* Politik ist daher gerade kein notwendiges Merkmal der Aufgabe, sondern ein Indikator für einen unvollständigen Belief-State; er wird in der Evaluation als solcher geprüft (Abschnitt 7). Stoneforge ist ein POMDP, da der Agent nur einen lokalen 15×15 -Ausschnitt sieht (siehe Abschnitt 4). Abbildung 3 stellt den Unterschied schematisch dar.

2.3 Wertbasierte Verfahren: Q-Learning und DQN

Wertbasierte Verfahren lernen die Aktionswertfunktion $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ und leiten daraus die Politik ab (Aktion mit dem höchsten Q -Wert). Q-Learning aktualisiert Q über die Bellman-Gleichung. Deep Q-Networks (DQN) approximieren Q mit einem neuronalen Netz und stabilisieren das Training über einen Replay-Buffer und ein Target-Netz [7]. DQN arbeitet nur mit diskreten Aktionsräumen; sein früher Einsatz im Projekt und der Wechsel zu PPO sind in Abschnitt 5 begründet.

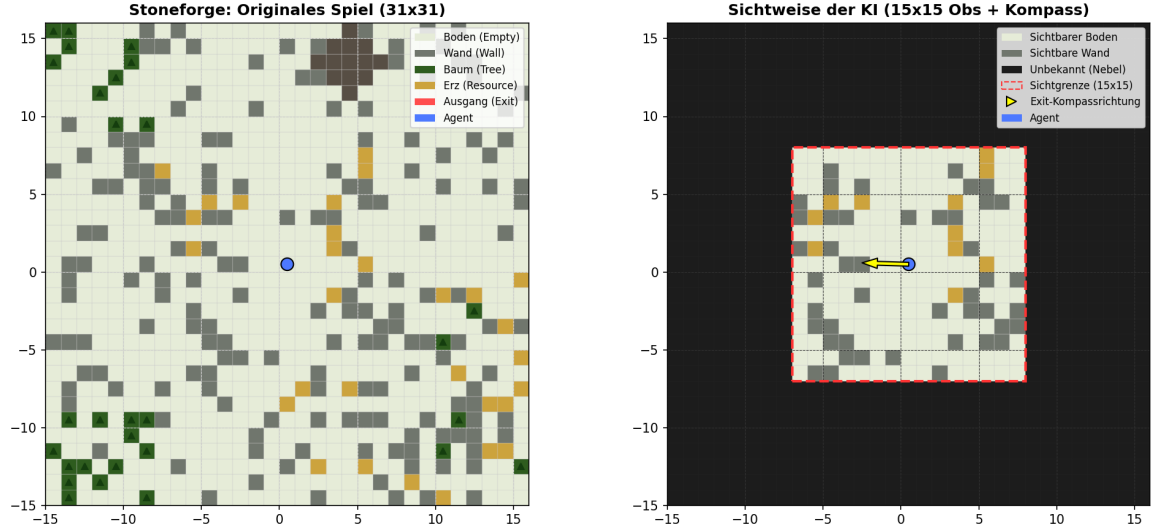


Abbildung 3: Partielle Beobachtbarkeit am Beispiel Stoneforge. Links der vollständige Weltzustand (31×31 -Ausschnitt), rechts die Beobachtung des Agenten: nur das rot umrandete 15×15 -Sichtfeld ist bekannt. Der Pfeil zeigt die Kompassrichtung zum Exit, nicht den Weg.

2.4 Policy-Gradient- und Actor-Critic-Verfahren

Policy-Gradient-Verfahren optimieren die Politik π_θ direkt über den Gradienten des erwarteten Returns. Actor-Critic-Verfahren kombinieren dies mit einer gelernten Wertfunktion (Critic), die als Baseline die Varianz der Gradientenschätzung senkt. Der Vorteil (Advantage) $A(s, a) = Q(s, a) - V(s)$ misst, wie viel besser eine Aktion gegenüber dem Durchschnitt ist; die Generalized Advantage Estimation (GAE) schätzt ihn über mehrere Zeitschritte mit einem Parameter λ [8] (im Projekt $\lambda = 0,95$). Advantage Actor-Critic (A2C) ist die synchrone Variante von A3C [9]. Wie gut der Critic gelernt hat, misst die *explained variance* (EV): der Anteil der Varianz der tatsächlichen Returns, den die Wertschätzung erklärt. $EV \approx 1$ bedeutet eine verlässliche Wertfunktion, $EV \approx 0$, dass der Critic praktisch nichts gelernt hat. Diese Größe dient in der Arbeit als zentrale Diagnosemetrik des Trainings.

2.5 Proximal Policy Optimization (PPO)

PPO ist das im Projekt eingesetzte Hauptverfahren [10]. Es ist ein Actor-Critic-Verfahren, das die Politik pro Update nur begrenzt verändert, um Instabilität durch zu große Schritte zu vermeiden. Dazu wird das Wahrscheinlichkeitsverhältnis $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{alt}}}(a_t | s_t)$ in der Zielfunktion abgeschnitten (*clipping*):

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] .$$

Abbildung 4 veranschaulicht die Wirkung des Clippings an einem einzelnen Zeitschritt. War die Aktion besser als erwartet ($A_t > 0$), lohnt es sich, ihre Wahrscheinlichkeit zu erhöhen, aber nur bis zum Faktor $1 + \epsilon$; darüber hinaus verläuft die Zielfunktion flach, der Anreiz für einen größeren Schritt entfällt. War sie schlechter als erwartet ($A_t < 0$), gilt dasselbe spiegelbildlich bei $1 - \epsilon$. Genau das verhindert die zu großen Aktualisierungen, an denen einfache Policy-Gradient-Verfahren scheitern.

Zusätzlich wird ein Entropie-Term addiert, der Exploration fördert; sein Gewicht (`ent_coef`) steuert, wie stochastisch die Politik bleibt. Trainiert wird in Zyklen: Erst sammeln die parallelen Umgebungen einen Rollout von n_{steps} Schritten, dann wird dieser in Minibatches der Größe `batch_size` über n_{epochs} Durchläufe optimiert. Verwendet wird die Implementierung aus Stable-Baselines3 [11].

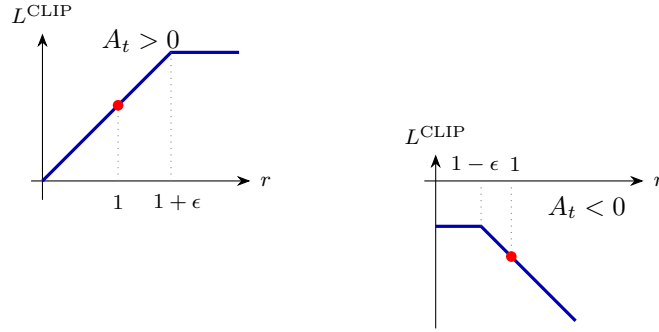


Abbildung 4: Ein einzelner Term der abgeschnittenen Zielfunktion L^{CLIP} über dem Wahrheitsverhältnis $r_t(\theta)$, links für positiven, rechts für negativen Advantage. Der rote Punkt markiert den Startpunkt der Optimierung ($r = 1$). Eigene Darstellung in Anlehnung an Schulman et al. [10, Abb. 1].

Eine gelernte stochastische Politik lässt sich auf zwei Arten ausführen: *stochastisch* (die Aktion wird aus der Verteilung $\pi(a | s)$ gezogen, Sampling) oder *deterministisch* (stets die wahrscheinlichste Aktion, Argmax). Dasselbe trainierte Modell kann in beiden Betriebsarten unterschiedlich gut abschneiden. Der Abstand zwischen beiden Messwerten wird in dieser Arbeit durchgängig als **Det/Stoch-Gap** bezeichnet; er ist einer ihrer zentralen Untersuchungsgegenstände, weil er sich als groß und hartnäckig erwiesen hat (Abschnitt 7.5). Anschaulich gesprochen: Der Agent löst die Aufgabe zuverlässiger, wenn er würfelt, als wenn er stets seine eigene beste Aktion wählt. Dass dies unter partieller Beobachtbarkeit möglich ist, folgt aus Abschnitt 2.2; dass es bei einer Politik *mit* Gedächtnis dennoch erklärungsbedürftig bleibt, ebenfalls.

2.6 Rekurrente Politiken (LSTM)

Alle bisher genannten Verfahren lassen sich mit einem gewöhnlichen vorwärtsgerichteten neuronalen Netz umsetzen, einem *Multilayer Perceptron* (MLP). Ein solches Netz bildet jede Beobachtung unabhängig auf eine Aktion ab und besitzt kein Gedächtnis: Es kann nicht unterscheiden, ob es eine Stelle zum ersten oder zum zehnten Mal sieht. Unter partieller Beobachtbarkeit ist das eine wesentliche Einschränkung (Abschnitt 2.2); die Bezeichnung *MLP* steht im Folgenden durchgängig für diese gedächtnislose Variante.

Um in einem POMDP ein Gedächtnis bereitzustellen, wird die Politik daher um ein rekurrentes Netz erweitert. Long Short-Term Memory (LSTM) ist eine rekurrente Architektur, die Information über viele Zeitschritte hinweg speichern kann [12]. Der LSTM-Zustand approximiert den Belief-State und fasst die bisher gesehenen Beobachtungen zusammen. Die Kombination aus rekurrentem Netz und wertbasiertem RL wurde als DRQN eingeführt [5]; im Projekt wird die policy-gradient-Variante RecurrentPPO (PPO mit LSTM-Politik) aus *sb3-contrib* verwendet, mit einer LSTM-Größe von 256.

Wie das Gedächtnis wirkt, zeigt Abbildung 5 in der über die Zeit „entrollten“ Darstellung: Der verborgene Zustand h_t wird von Schritt zu Schritt weitergereicht und sammelt so Information, die in der einzelnen Beobachtung nicht enthalten ist. Für Stoneforge ist das genau die Information „hier war ich schon“ bzw. „diese Wand bin ich bereits nach Norden abgelaufen“.

2.7 Reward Shaping (PBRs)

Zusätzliche Belohnungen können das Lernen beschleunigen, dürfen die optimale Politik aber nicht verändern. Potential-Based Reward Shaping (PBRs) erfüllt dies: Der Zusatzreward hat

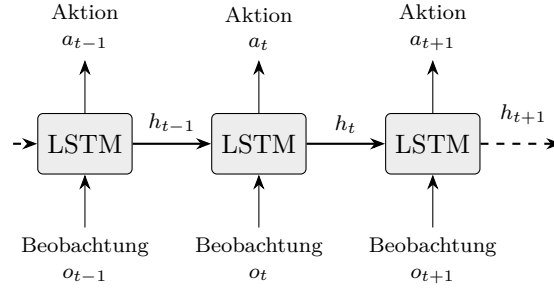


Abbildung 5: Über die Zeit entrollte rekurrente Politik: Neben der aktuellen Beobachtung o_t erhält jeder Zeitschritt den verborgenen Zustand h_{t-1} des Vorschritts. Eigene Darstellung; rekurrente Politiken im RL nach Hausknecht und Stone [5].

die Form $F(s, s') = \gamma \Phi(s') - \Phi(s)$ mit einer Potentialfunktion Φ . Ng et al. zeigen, dass eine solche Form policy-invariant ist, die optimale Politik also erhalten bleibt [13]. Im Projekt ist Φ an die BFS-Distanz zum Exit gekoppelt: Die Breitensuche (BFS) liefert die Länge des kürzesten begehbaren Wegs vom Agenten zum Exit in Schritten. Im Gegensatz zur Luftlinie berücksichtigt sie Wände und Umwege.

2.8 Curriculum Learning

Curriculum Learning bezeichnet das Training mit ansteigender Aufgabenschwierigkeit statt einer zufälligen Reihenfolge [14]. Im Projekt wird die Exit-Distanz stufenweise erhöht; der Übergang zwischen den Stufen erfolgt leistungsbasiert (eine Stufe gilt ab einer Ziel-Erfolgsquote als bestanden).

2.9 Prozedurale Generierung und Generalisierung

Wird für jedes Level eine neue, zufällig erzeugte Welt verwendet, kann der Agent einzelne Karten nicht auswendig lernen und muss allgemeine Strategien bilden. Die Erzeugung ist über einen *Seed* steuerbar, den Startwert des Zufallszahlengenerators: Derselbe Seed erzeugt reproduzierbar dieselbe Welt, verschiedene Seeds verschiedene Welten. Prozedurale Generierung ist daher ein etabliertes Mittel, um Generalisierung in RL zu prüfen und Overfitting auf einzelne Level zu vermeiden [1, 2]. Die Evaluation auf trainingsfremden Seeds (Testset A und Holdout B) folgt diesem Prinzip.

3 Einordnung in verwandte Arbeiten

Stoneforge liegt strukturell zwischen drei etablierten Benchmarks: MiniGrid (teilbeobachtbare Grid-Navigation, PPO+LSTM als Standard-Baseline) [15], Crafter (prozedural generiertes 2D-Survival-Spiel mit Mining und Crafting) [16] und Procgen (Generalisierung über prozedurale Level) [1]. Prozedurale Generierung als Mittel, Generalisierung zu erzwingen, hat darüber hinaus eine eigene Forschungstradition [17, 2].

Gedächtnis in teilbeobachtbaren Umgebungen. Die Standardantwort auf partielle Beobachtbarkeit ist eine rekurrente Politik: DRQN erweiterte DQN um ein LSTM [5]; R2D2 zeigte, dass die korrekte Behandlung des rekurrenten Zustands (gespeicherte States im Replay, Burn-in) die Leistung bei ansonsten unveränderter Architektur erheblich beeinflusst [18]. Dieser Befund spiegelt sich in dieser Arbeit auf der Eval-Seite wider (korrekt geführter LSTM-State, Abschnitt 5). Für on-policy-Verfahren untersuchen Pleines et al. Implementierungsdetails rekurrenter PPO-Politiken und stellen Umgebungen vor, deren Gedächtnis

sanforderungen über reine Kapazität hinausgehen [19]. Der POPGym-Benchmark vergleicht 13 Gedächtnis-Architekturen systematisch; dort schneidet das einfache GRU insgesamt am besten ab; aufwändigere Gedächtnismodelle sind nicht automatisch besser [20]. Der hier verwendete RecurrentPPO-Ansatz entspricht dieser Standard-Baseline, und der Befund der eigenen E3-Ablation (LSTM 512 schlägt 256 nicht, Abschnitt 7.5) fügt sich in dieses Bild. Strukturierte Alternativen wie das In-Context-Verfahren AMAGO [21] werden im Ausblick diskutiert.

Curriculum und Exploration. Curriculum Learning geht auf Bengio et al. zurück [14]; das hier verwendete leistungsorientierte Gating ist eine einfache Instanz, Prioritized Level Replay (PLR) [22] die naheliegende Weiterentwicklung auf Level-Ebene. PLR wiederholt gezielt jene Level, an denen der Agent noch am meisten lernen kann, statt sie gleichverteilt zu ziehen. Das Training dieser Arbeit enthält eine stark vereinfachte Variante dieses Gedankens: einen *Seed-Pool*, der einen Teil der Episodenstarts durch bereits gelöste Welten ersetzt (Abschnitt 6). Wie in Abschnitt 10.5 diskutiert, kehrt diese Variante die Auswahlrichtung von PLR gerade um und ist deshalb kritisch zu sehen. Der Explorations-Bonus für neu betretene Tiles ist eine zählbasierte Form intrinsischer Motivation; curiositätsgetriebene Verfahren wie ICM [23] und RND [24] verallgemeinern dieses Prinzip auf Beobachtungsräume, in denen Neuheit nicht direkt abzählbar ist. Dies ist hier nicht erforderlich, da die Tile-Position Neuheit unmittelbar definiert.

Zur Vergleichbarkeit der absoluten Zahlen. Ein direkter Vergleich der hier erreichten Erfolgsquote mit publizierten Werten dieser Benchmarks ist *nicht* seriös möglich, weil sie unterschiedliche Metriken berichten: Procgen aggregiert über den *normalisierten Return* $(R - R_{\min}) / (R_{\max} - R_{\min})$ und nicht über eine Erfolgsquote [1]; Crafter bewertet erreichte *Achievements* [16]; für MiniGrid-Aufgaben berichten Folgearbeiten aufgabenspezifische Erfolgsquoten, die je nach Aufgabe bis 100 % reichen. Hinzu kommen Unterschiede in Sichtradius, Episodenlänge und Zielabstand. Eine Aussage der Form „unser Wert liegt über/unter dem Stand der Technik“ wäre daher eine Scheinpräzision und wird hier bewusst nicht getroffen.

Belastbar ist der Vergleich nur *innerhalb* dieser Arbeit: gegen die eigene Gegenüberstellung (Tabelle 8), gegen die Gap-Experimente (Abschnitt 7.5) und gegen die eigenen, vorab fixierten Kriterien (Abschnitt 5.5). Alle Vergleichsbedingungen laufen dort auf demselben Generator, mit derselben Metrik und demselben Eval-Protokoll.

4 Die Umgebung Stoneforge

Stoneforge ist ein selbst entwickeltes, rundenbasiertes 2D-Spiel (C++-Kern, Python-Anbindung über pybind11). Die Welt wird pro Seed deterministisch aus Hash-Noise generiert (7 Biome, Chunks à 16×16 Tiles, theoretisch unendlich); der Agent startet bei (0,0) und muss den Exit erreichen. Abbildung 6 zeigt die Benutzeroberfläche und die Spielwelt aus Sicht eines menschlichen Spielers.

4.1 POMDP-Charakter

Der Agent sieht nur einen lokalen 15×15-Ausschnitt der Welt sowie einen *Luftlinien*-Kompass zum Exit, nicht den Weg. Wände, Sackgassen und Umwege außerhalb des Sichtfensters sind unbekannt: Die Aufgabe ist ein *Partially Observable Markov Decision Process* (POMDP, Abschnitt 2.2). Beide Perspektiven stellt Abbildung 3 auf Seite 5 gegenüber: links die Welt, wie sie tatsächlich aussieht, rechts das, was der Agent davon sieht. Für eine gedächtnislose Politik sind dabei verschiedene Weltpositionen ununterscheidbar (Abschnitt 7.1).



Abbildung 6: Benutzeroberfläche des spielbaren C++-Clients (Stoneforge 2D, echter Screenshot): Gezeigt ist die Spielumgebung aus Sichtweise eines menschlichen Spielers mit vollem Sichtbereich, dem Status-Header (Lebenspunkte, Energie, Schritte) und der Spielfigur im Zentrum.

4.2 Beobachtung, Aktionen, Episodenende

Bereich	Inhalt	Dim.
Lokales Grid	15×15 Tile-Typen um den Agenten (Sichtradius 7)	225
HP	Lebenspunkte (normiert)	1
Kompass	exitDx, exitDy (Luftlinie, normiert)	2
Episodenfortschritt	Schritt / 4000	1
Gesamt (Env v11)		229

Tabelle 2: Observation der Umgebungsversion v11. Die früheren Features *Energie* und *Inventory* waren im RL-Setup konstant (tote Features) und wurden am 06.07.2026 entfernt (vorher 231 Dimensionen).

Der Aktionsraum ist **Discrete(4)**: hoch, runter, links, rechts. Mining, Bauen und Kampf existieren nur im spielbaren Client und sind aus dem RL-Pfad vollständig entfernt. Eine Episode endet bei Erreichen des Exits (Sieg), nach 4000 Schritten (Timeout) oder als Truncation nach 256 Schritten ohne positiven Reward (Early-Stop).

4.3 Reward-Design

Das Reward-Design folgt dem Prinzip: *sparse Primärsignal + theoretisch sauberes dichtes Shaping*. Kernstück ist Potential-Based Reward Shaping (PBRS) auf der BFS-Pfaddistanz zum Exit:

$$F(s, s') = \gamma \Phi(s') - \Phi(s), \quad \Phi(s) = -\frac{d_{\text{BFS}}(s)}{128}, \quad (1)$$

mit $\gamma = 0.999$ (identisch zum RL-Discount) und Gewicht $\beta = 2.5$. Ein Tile echter Fortschritt liefert damit rund $+0,02$; abzüglich des Schritt-Malus von $-0,01$ (Tabelle 3) verbleiben netto etwa $+0,01$. PBRS ist beweisbar *policy-invariant*; es beschleunigt die Konvergenz, ohne die optimale Politik zu verändern [13]. Entscheidend ist, dass das Potential die *echte* Pfaddistanz

(BFS durch Wände gerechnet) verwendet, nicht die Luftlinie: Ein Luftlinien-Potential würde den Agenten systematisch in Sackgassen führen.

Diese Garantie gilt allerdings nur für den PBRs-Term selbst. Die übrigen Komponenten in Tabelle 3 (Explorations-Bonus, Schritt-, Loop-, Idle- und Schadens-Penalty) sind gewöhnliche Reward-Terme und verändern die optimale Politik prinzipiell mit. Der frühere eskalierende Wand-Penalty ($-0.05/-0.25$) wurde aus genau diesem Grund entfernt: Die Summe vieler kleiner Strafen machte Erkunden in engen Korridoren unwirtschaftlicher als Stillstand.

Komponente	Wert
Exit erreicht (terminal)	+100
Timeout / Tod (terminal)	-10
PBRs (BFS-Distanz, pro Schritt)	$\approx \pm 0.02$
Explorations-Bonus (neue Tile)	+0.02
Schritt-Penalty	-0.01
2-Schritt-Loop-Penalty	-0.05
Idle-Penalty (Aktion ohne Positionswechsel)	-0.04
Schadens-Penalty (pro verlorenem HP)	-0.5

Tabelle 3: Vollständige Reward-Komponenten der Umgebungsversion v11 gemäß `Simulation::computeReward()`.

4.4 Garantierte Lösbarkeit

Die Exit-Platzierung wählt Kandidaten per Flood-Fill vom Spawn aus ausschließlich über *begehbare* Tiles; jeder mögliche Exit ist damit per Konstruktion erreichbar. Empirisch bestätigt: 3 150 geprüfte Seeds (inkl. aller Eval-Sets und Trainingsbereiche) sowie eine erneute Messung am 05.07.2026 (150/150) ergaben **100 % Lösbarkeit**. Ein BFS-Oracle-Agent erreicht den Exit auf allen Test-Seeds in exakt der BFS-Distanz und bestätigt damit die gesamte Kette aus Weltgenerierung, Bewegungsmechanik und Reward-Signal.

5 Methodik

5.1 Algorithmus

Trainiert wird RecurrentPPO (PPO [10] mit LSTM-Politik [12]; beide Verfahren in Abschnitt 2) aus *Stable-Baselines3* / *sb3-contrib* [11]. Zentrale Hyperparameter: $n_{\text{steps}} = 256$, $n_{\text{epochs}} = 10$, Lernrate $3 \cdot 10^{-4}$, $\gamma = 0.999$, `ent_coef`=0.05, LSTM mit 256 Hidden Units (separater Critic-LSTM), 16 parallele Umgebungen.

Algorithmenwahl. Die Trainings-Infrastruktur unterstützt neben PPO auch DQN und A2C. Ein früher DQN-Anlauf (Mai 2026) wurde nach Diagnose verworfen: Die Q-Werte kollabierten auf ein nahezu aktionsunabhängiges Niveau, und der Replay-Buffer konservierte die Folgen eines später behobenen Reward-Fehlers über das weitere Training. Ein on-policy-Verfahren ist gegenüber solchen Fehlerfolgen strukturell robuster, weshalb auf PPO gewechselt wurde (Anhang, Etappe 1). Nach der Umstellung auf die rekurrente Politik wurde kein wertbasiertes Verfahren erneut vermessen; eine quantifizierte DQN/A2C-Baseline auf der finalen Umgebung v11 steht damit aus und wird als Grenze der Arbeit ausgewiesen (Abschnitt 8).

5.2 Curriculum

Das Training verläuft leistungsbasiert in vier Phasen steigender Exit-Distanz; eine Phase endet, wenn die Ziel-Erfolgsquote in zwei aufeinanderfolgenden Evaluationen erreicht wird.

Phase	Exit-Distanz	Ziel-SR	Gating-Metrik	ent_coef
1	5–12	85 %	stochastisch	0.05
2	12–25	70 %	stochastisch	0.05
3	25–45 (Eval 35–45)	70 %	deterministisch	0.05 $\rightarrow$ 0.001 (Annealing)
4	25–45 (Greedy Fine-Tune)	60 %	deterministisch	0.0001

Tabelle 4: Curriculum (Stand v11). Phase 1/2 gaten auf der stochastischen SR, da die Politik dort mit `ent_coef=0.05` absichtlich nahe-uniform ist und die deterministische SR konstruktionsbedingt kein Signal trägt.

Das Distanz-Curriculum staffelt dabei implizit auch die *Beobachtbarkeit* des Ziels, und damit die Fähigkeit, die je Phase gelernt werden muss. Eine Messung über 200 Welten pro Phase (07.07.2026): In Phase 1 liegt der Exit beim Episodenstart in **72 %** der Welten bereits im 15×15 -Sichtfeld (zu lernen: „geh zum sichtbaren Ziel, weiche lokalen Hindernissen aus“); in Phase 2 nur noch in **6 %** (zu lernen: dem Kompass folgen, Umwege um Hindernisse); in Phase 3 in **0 %** (reine Kompass-Navigation über lange Distanz, inklusive Sackgassen, die nur mit Gedächtnis überwindbar sind). Ein zusätzliches Curriculum über die Sichtfeldgröße (in der Literatur als Übergang von voller zu partieller Beobachtbarkeit beschrieben) wäre daher redundant; zudem wäre es hier riskant, weil ein großes Sichtfeld anfangs eine *reaktive* Strategie ohne Gedächtnisnutzung belohnt, die beim Verkleinern des Fensters wertlos wird (dieselbe Nicht-Übertragbarkeit, die die Gegenüberstellung in Abschnitt 7.1 für das BFS-Orakel zeigt).

5.3 Iterative Modellverbesserung

Die finale Konfiguration ist nicht das Ergebnis einer einmaligen Wahl, sondern systematischer Fehlersuche über zwölf Umgebungs- bzw. Trainingsiterationen; der vollständige Verlauf inklusive aller Fehlversuche ist im Anhang dokumentiert (Abschnitt 10). Drei Befunde waren entscheidungsprägend:

1. **Hyperparameter dominieren Features.** Eine Serie von Feature-Erweiterungen (v6–v9) scheiterte vollständig (SR 0–12 %), und zwar nicht an den Features, sondern an nebenbei verstellten Basis-Hyperparametern; dominanter Faktor war `ent_coef` (0,05 $\rightarrow$ 0,01: zu wenig Exploration, der Exit wird nie gefunden). Aufgeklärt durch Reverse-Engineering des letzten funktionierenden Modells (Tabelle 14).
2. **Einzel-Snapshots validieren nicht.** Die Batch-Größe 64 war zunächst durch einen einzelnen Messpunkt „validiert“; die vollständige Eval-Kurve zeigte jedoch eine starke Oszillation ohne Konvergenz (`Critic-explained_variance` $\approx$ 0,1). Mit `batch_size=8` ($\approx 8 \times$ mehr Gradientenschritte pro Rollout) stieg die EV über 0,85 bei stabilem Lernen (Abschnitt 10.8.2). Hyperparameter-Entscheidungen werden seitdem nur auf ganzen Eval-Kurven getroffen.
3. **Das Eval-Protokoll ist Teil des Experiments.** Ein Episoden-Cap von 600 statt 4 000 maß dasselbe Modell mit 48 % statt 86 % SR und machte ein Phasenziel unerreichbar (Abschnitt 10.6); ebenso verzerrte ein Data-Leakage über den Eval-Callback (Modellauswahl auf Test-Seeds) die Selektion, bis am 11.06. das separate Validierungsset eingeführt wurde (Anhang, Etappe 2).

Voraussetzung dieser Analysen war die lückenlose Protokollierung aller Experimente im Changelog; ohne sie wäre insbesondere das Reverse-Engineering in Befund 1 nicht möglich gewesen.

5.4 Evaluationsprotokoll

Zentrale Metrik ist die *Erfolgsquote* (*Success Rate*, SR): der Anteil von 50 festen, trainingsfremden Welten, in denen der Agent den Exit innerhalb des Episodenlimits (4000 Schritte) erreicht. Ein zu kurzes Limit verfälscht die SR erheblich (dasselbe Modell: 48 % bei Limit 600 gegenüber 86 % bei 4000; Abschnitt 10.6), daher wird stets mit dem vollen Limit gemessen.

Strikte Seed-Trennung: Validierung (6000–6049) steuert Phasenwechsel und Modellauswahl; Testset A (7000–7049) und Holdout B (8000–8049) werden ausschließlich final evaluiert. Ein früherer Data-Leakage-Fehler (Modellauswahl auf den Test-Seeds) wurde identifiziert und behoben. Jede Evaluation misst seit v11 *beide* Betriebsarten: stochastisch (Sampling aus der Politik) und deterministisch (Argmax). Bei der rekurrenten Politik werden LSTM-Zustand und Episoden-Start-Signal an jeden `predict()`-Aufruf durchgereicht und zu Episodenbeginn zurückgesetzt, damit kein Zustand über Episodengrenzen übertragen wird; dies ist ein häufiger, leicht übersehener Fehler beim Evaluieren rekurrenter Politiken. Für das Endergebnis werden sieben unabhängige Läufe gemittelt (Mittelwert $\pm$ Stichproben-Standardabweichung, $\text{ddof} = 1$, zusätzlich 95 %-Konfidenzintervalle); als Trainings-Diagnosemetrik dient die `explained_variance` des Critics. Ergebnisse in Abschnitt 7.

5.5 Herleitung der Erfolgsschwellen

Die folgende Herleitung setzt Kenntnis der Umgebung voraus und steht deshalb hier und nicht bei der Nennung der Kriterien in Abschnitt 1.4.

Die dort genannten Schwellen (70 % auf Testset A, 60 % auf Holdout B) ersetzen die im Exposé angesetzte Metrik von $\geq 90\%$ über 100 unbekannte Seeds. Drei Eigenschaften der Aufgabe, die zu Projektbeginn noch nicht bekannt waren, machen einen Wert in dieser Höhe unerreichbar:

1. **Der POMDP-Charakter der Umgebung.** Bei einem Sichtradius von 7 und nicht sichtbarem Ziel ist die Aufgabe partiell beobachtbar (Abschnitt 2.2). Eine deterministische Politik über der *Beobachtung* ist damit nicht mehr garantiert optimal [6], hinzu kommt die epistemische Unsicherheit über den Seed [25]. Eine Schwelle von 90 % unterstellt dagegen eine nahezu vollständig lösbare Aufgabe.
2. **Die tatsächliche Schwierigkeit der Distanzvorgabe.** Bis zur Umgebungsrevision v11 wurde die Exit-Distanz als Luftlinie gemessen; eine Vorgabe von „35–45“ entsprach real einem Laufweg von 42–75 Feldern (Abschnitt 10.7).
3. **Die Streuung über Trainingsläufe.** Sie beträgt rund ± 12 Prozentpunkte (Tabelle 9). Eine Schwelle von 90 % liegt weit außerhalb dessen, was das Verfahren hier stabil liefert.

Die Schwellen 70 %/60 % sind seit dem 16.05.2026 im Experiment-Changelog festgehalten, also lange vor den finalen Läufen (08.07.) und der $n = 7$ -Auswertung (17.07.); sie wurden nicht in Kenntnis der hier berichteten Ergebnisse gewählt und sind damit kein Fall von *HARKing* (*Hypothesizing After the Results are Known* [26]). Dass das Testset-Kriterium am Ende dennoch verfehlt wird, zeigt, dass auch die revidierte Schwelle keine bequeme war. Gegen die ursprüngliche Exposé-Metrik gemessen ergeben Testset A und Holdout B zusammen die geforderten 100 unbekannten Seeds und einen Mittelwert von $66,3\% \pm 12,1$ (stochastisch); die 90 %-Schwelle wird deutlich verfehlt. Das Exposé nennt als zweite Zielgröße die Minimierung der *Steps per Episode*. Auch sie wird verfehlt, und zwar deutlicher als die Erfolgsquote: Die in Abschnitt 7.3 berichtete Pfadefizienz von $\eta \approx 0,05$ entspricht rund dem Zwanzigfachen des kürzesten Weges.

5.6 Versuchsübersicht: Setup je Iteration

Die folgenden Angaben zum Setup jeder systematischen Iteration folgen den Reproduzierbarkeits-Checklisten von Pineau et al. [27] bzw. Henderson et al. [28]. Alle Läufe teilen die in Tabelle 5 gelistete Basiskonfiguration; die Iterationen in Tabelle 6 verändern jeweils einen *inhaltlichen* Faktor gegenüber der v12-Baseline. Sie sind allerdings weder vollständig kontrolliert (E1/E2 laufen zusätzlich mit reduziertem Phase-3-Budget) noch repliziert ($n = 1$ je Arm); die daraus ableitbaren Schlüsse sind entsprechend schwach und werden in Abschnitt 7.5 eingegrenzt. Die vollständige Historie inklusive Zwischenständen liegt im Experiment-Changelog (CHANGELOG.md); je Lauf existiert eine `config.json` mit den hier tabellierten Werten.

Komponente	Wert
Algorithmus	RecurrentPPO (SB3-contrib), MlpLstmPolicy
n_{steps} / <code>batch_size</code> / n_{epochs}	256 / 8 / 10
Lernrate / γ / λ_{GAE} / clip	$3 \cdot 10^{-4}$ / 0.999 / 0.95 / 0.2
<code>ent_coef</code>	0.05 (Phase 3: Annealing auf 0.001; Phase 4: 0.0001)
<code>vf_coef</code>	0.5
LSTM	1 Schicht, 256 Hidden Units, separater Critic-LSTM
Beobachtung	229 Features (Grid 15×15 + HP + Kompass + Step-Frac), Env v11
Parallele Umgebungen	16 (DummyVecEnv), Device CPU
Seed-Pool („Swarm“)	aktiv, Sampling-Wahrscheinlichkeit 0.3
Curriculum	4 Phasen (Tab. 4), leistungs basiert
Eval	Testset A 7000–7049, Holdout B 8000–8049, Cap 4000, det + stoch

Tabelle 5: Gemeinsame Basiskonfiguration aller Iterationen (Quelle: `config.json` der v12-Läufe).

Iteration	Änderung ggü. Baseline	Start	Test A (stoch/det)	Holdout B (stoch/det)
v12 (Baseline, $n=3$)	– (LSTM 256, <code>vf_coef</code> 0.5)	Phase 1	73/32 %	80/42 %
E1 critic	<code>vf_coef</code> 0.5 $\rightarrow$ 1.0	Phase 3	56/42 %	72/36 %
E2 curric	Phase 3 Exit 25–45 $\rightarrow$ 25–35	Phase 3	26/16 %	42/24 %
E3 lstm512	LSTM 256 $\rightarrow$ 512	Phase 1	44/14 %	58/20 %

Tabelle 6: Iterationen zur Schließung des Det/Stoch-Gaps; jeder Arm ein Einzellauf, die Baseline ein Mittel aus drei Läufen. Werte standardisiert (50 Seeds, Limit 4 000). Zur Aussagekraft siehe Abschnitt 7.5.

Zwei Einschränkungen sind beim Lesen der Tabelle zu beachten. E1 und E2 setzen aus einem gemeinsamen Phase-2-Checkpoint auf und erhalten nur 400k statt 1,0 Mio. Phase-3-Schritte; sie variieren gegenüber der Baseline also zwei Größen und nicht eine. E3 ist dagegen ein voller Lauf ab Phase 1. Da zudem jeder Arm nur einmal gelaufen ist, die Streuung über Läufe aber rund ± 12 Punkte beträgt, sind Abstände unterhalb von etwa 25 Punkten nicht interpretierbar.

Ergänzend zeigt die Fehleranalyse in Tabelle 14 die frühere Generationenreihe (v2/v10 Referenz, v7–v9 fehlgeschlagen), deren Setup sich ausschließlich in den dort genannten vier Parametern unterscheidet.

6 Technische Umsetzung und Implementierung

6.1 Technologie-Stack

Das System besteht aus drei Schichten: einem C++-Kern für Simulation und Weltgenerierung, einem Python-Binding und dem Python-Trainingsstack. Tabelle 7 listet die verwendeten Komponenten.

Komponente	Technologie	Zweck
Simulationskern	C++20 (ca. 4 500 LOC)	Weltgenerierung, Spiellogik, Reward, BFS
Binding	pybind11 2.13	C++-Kern als Python-Modul (<code>stoneforge_sim</code>)
Build	CMake	Kern, Binding, Clients
RL-Umgebung	Python 3.12, Gymnasium 1.2	<code>StoneforgeWorldEnv</code> (Gym-API)
RL-Algorithmen	Stable-Baselines3 / sb3-contrib 2.8	PPO, DQN, A2C, RecurrentPPO
Deep Learning	PyTorch 2.12 (CPU)	Netz- und LSTM-Training
Spiel-Client	raylib 5.5 (C++, ca. 4 500 LOC)	spielbarer Client, KI-Zuschaumodus
Monitoring	TensorBoard, WebSockets	Trainingsmetriken, Live-Visualisierung
Auswertung	NumPy, Matplotlib	Evaluation, Abbildungen

Tabelle 7: Technologie-Stack.

Die Simulation ist deterministisch pro Seed. Training und Evaluation laufen vollständig auf der CPU; ein Vergleich mit der GPU ergab für dieses kleine Netz keine Beschleunigung. Der Durchsatz liegt je nach Konfiguration bei ca. 90–190 Umgebungsschritten pro Sekunde. Die verwendete Hardware und der zugehörige Benchmark sind in Anhang 10.8 dokumentiert.

6.2 Umgebung und Binding

`StoneforgeWorldEnv` (Python) kapselt den C++-Kern hinter der Gymnasium-API. Die Beobachtung umfasst 229 Merkmale (Grid 15×15 als Tile-Typen, HP, Exit-Kompass dx/dy , Schrittteil); der Aktionsraum ist `Discrete(4)` (vier Bewegungsrichtungen). Die im Spiel vorhandenen Aktionen Mining, Bauen und Kampf sind im RL-Binding deaktiviert und werfen bei Aufruf einen Fehler; sie existieren nur im spielbaren Client. Eine Episode endet bei Erreichen des Exits, nach 4 000 Schritten oder als Early-Stop-Truncation nach 256 Schritten ohne positiven Reward (Abschnitt 4).

Zwei Implementierungsdetails waren fehlerkritisch: (1) Die Weltgenerierungs-Konfiguration ist im C++-Kern prozessglobal; jede Env-Instanz stempelt sie deshalb bei jedem `reset()` neu, da sonst parallel erzeugte Evaluations-Umgebungen die Trainingsverteilung verändern (Abschnitt 10.7). (2) Der Exit wird seit v11 nach realer BFS-Laufwegdistanz platziert statt nach Luftlinie; alle erzeugten Welten sind nachweislich lösbar (150/150 im Test).

Der Reward setzt sich zusammen aus Schritt-Malus (−0,01), Explorations-Bonus (+0,02 pro neuem Tile), Loop-Malus (−0,05), Schadens-Malus, dem PBRs-Term auf der BFS-Distanz (Abschnitt 2) und dem Terminal-Reward.

6.3 Trainings-Pipeline

Das Curriculum-Training (`train_curriculum.py`) orchestriert die vier Phasen aus Tabelle 4. Je Lauf arbeiten 16 parallele Umgebungen (`DummyVecEnv`). Alle 25 000 Schritte evaluiert ein Callback das aktuelle Modell auf den 50 Validierungs-Seeds in beiden Betriebsarten; die Phasen-Metrik steuert Gating (Ziel-SR in zwei aufeinanderfolgenden Evaluationen) und die Auswahl des besten Modells der Phase. Beim Phasenwechsel wird das beste Modell der Vorphase geladen. In Phase 3 senkt ein Callback den Entropie-Koeffizienten linear von 0,05 auf 0,001 über 500 000 Schritte; Phase 4 trainiert mit 0,0001 (Greedy Fine-Tune). Optional ergänzt ein Seed-Pool („Swarm“) 30 % der Episodenstarts durch Wiederholung bereits

gelöster Seeds; der Pool wird bei jedem Phasenwechsel geleert. Jeder Lauf legt pro Phase Modell-Checkpoints sowie `eval_history.json` und `config.json` ab; alle Experimente sind zusätzlich im `CHANGELOG.md` des Repositories dokumentiert.

6.4 Monitoring und Werkzeuge

Trainingsmetriken werden nach TensorBoard geschrieben. Neben der bereits eingeführten `explained_variance` (Güte der Wertfunktion, Abschnitt 2) und der Entropie der Politik ist dabei `approx_kl` von Bedeutung: ein Schätzwert dafür, wie stark sich die Politik durch ein Update verändert hat. Werte nahe null zeigen an, dass das Training faktisch stillsteht; sehr große Werte deuten auf zu grobe Schritte hin. Die Größe dient in dieser Arbeit als schneller Test, ob ein Lauf überhaupt lernt (Anhang 10). Zusätzlich streamt ein WebSocket-Server den Trainingszustand in eine Browser-Ansicht: Erfolgsquoten-Kurven (deterministisch und stochastisch), Minimaps der 16 parallelen Agenten und eine Explorations-Heatmap. Eine Launcher-GUI (Tkinter) bündelt Training, Evaluation und Abspielen. Zum Vorführen steuert `demo_agent.py` den grafischen Client: Der Client läuft im KI-Modus und erhält die Aktionen des geladenen Modells über die Standardeingabe; Client und Trainings-Umgebung erzeugen dieselbe Welt aus demselben Seed.

7 Evaluation

Das Evaluationsprotokoll (Seed-Trennung, Metrik, Betriebsarten) ist in Abschnitt 5 beschrieben. Die Ergebnisse sind nach Beweislast geordnet: Der Vergleich gegen ungelernete Referenzpolitiken (Abschnitt 7.3) schränkt die Aussagekraft aller vorher berichteten Zahlen ein und steht deshalb nach ihnen.

7.1 Zentrale Gegenüberstellung: Gedächtnis statt lokalem Orakel

Bedingung	Arch.	BFS in Obs	Curric.	Obs-Dim.	SR stoch.	SR det.
A (ppo_phase4)	MLP	ja	ja	236	32 %	0 %
B (ppo_no_bfs)	MLP	nein	nein	230	–	≈ 0 %
C (ppo_lstm_curriculum)	LSTM	nein	ja	231	86 %	36 %

Tabelle 8: Gegenüberstellung dreier Modelle auf Testset A (Exit 35–45), Stand Juni 2026; je ein Einzellauf. Zur eingeschränkten Aussagekraft siehe den folgenden Absatz.

Aussagekraft. Diese Gegenüberstellung ist *keine* kontrollierte Ablation. Vier Einschränkungen begrenzen, was aus ihr folgt. Erstens unterscheiden sich A und C in *zwei* Faktoren gleichzeitig (Architektur *und* BFS-Features), der Effekt ist also nicht separierbar. Zweitens laufen die drei Bedingungen auf unterschiedlichen Observationsräumen und Umgebungsständen, alle vor der v11-Revision; A wurde zudem mit `forceGuaranteedPath` und 236 Merkmalen trainiert und erreicht unter dem damaligen kurzdistanzigen Eval-Protokoll 100 % deterministisch, unter dem hier verwendeten (Exit 35–45) hingegen 0 %. Drittens ist B kein vollwertiger Vergleichsarm: Der Lauf wurde bei 720 k von 2 Mio. Schritten abgebrochen und enthielt einen Stagnations-Penalty, der später als schädlich erkannt und entfernt wurde. Viertens ist jede Bedingung ein Einzellauf, bei einer Streuung über Läufe von rund ± 12 Punkten (Tabelle 9).

Belastbar ist daher nur die schwache Aussage, dass eine rekurrente Politik *ohne* lokale BFS-Features eine Leistung erreicht, die eine gedächtnislose Politik *mit* diesen Features unter demselben Protokoll nicht erreicht hat.

Das finale Modell (v12, Mittel über sieben Läufe) erreicht auf Holdout B 66,9 % (stochastisch) und erfüllt damit das Holdout-Kriterium; auf Testset A wird die 70 %-Schwelle mit 65,7 % verfehlt (Abschnitt 7.2). Abbildung 7 zeigt den Trainingsverlauf dieses historischen Referenzlaufs; die Verläufe der sieben finalen v12-Läufe zeigt Abbildung 8.

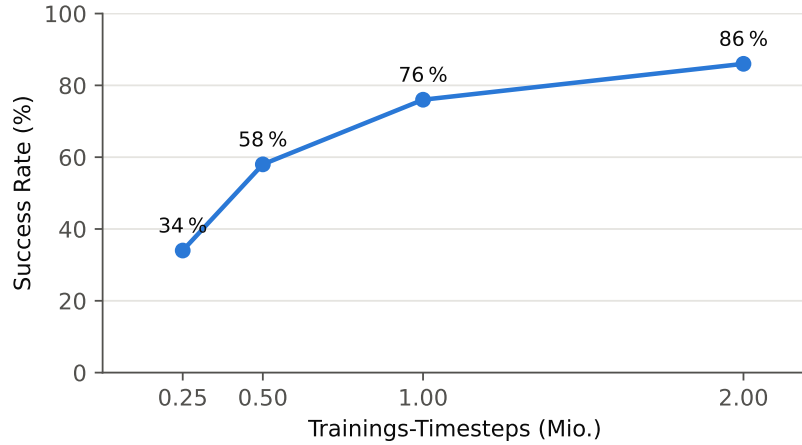


Abbildung 7: Lernkurve des historischen Referenzlaufs (ppo_lstm_curriculum, Bedingung C, Umgebung vor der v11-Revision; Curriculum-Phase 3, stochastische Evaluation auf 50 Seeds).

7.2 Endergebnis (v12, $n = 7$)

Die sieben finalen Curriculum-Läufe (alle vier Phasen durchlaufen) ergeben auf dem je Lauf nach Validierungs-SR besten Modell:

Lauf	A stoch.	A det.	B stoch.	B det.
Seed 1	62 %	38 %	76 %	46 %
Seed 2	84 %	26 %	76 %	44 %
Seed 3	76 %	32 %	80 %	36 %
Seed 4	74 %	40 %	74 %	38 %
Seed 5	58 %	20 %	62 %	34 %
Seed 6	50 %	20 %	50 %	14 %
Seed 7	56 %	28 %	50 %	16 %
Mittel $\pm$ Std	65,7 $\pm$ 12,4	29,1 $\pm$ 8,0	66,9 $\pm$ 12,8	32,6 $\pm$ 12,7
95 %-CI	[54,2; 77,2]	[21,8; 36,5]	[55,0; 78,7]	[20,8; 44,4]

Tabelle 9: Endergebnis der sieben v12-Läufe auf Testset A und Holdout B (je 50 Seeds, Limit 4000, beide Betriebsarten). Standardabweichung als Stichproben-Std (ddof = 1), Konfidenzintervall über die t -Verteilung (df = 6).

Ausgewertet wird je Lauf nicht der letzte Checkpoint, sondern das nach Validierungs-Erfolgsquote beste Modell. Die Auswahl erfolgt ausschließlich auf den disjunkten Seeds 6000–6049. Da über einen Lauf hinweg mehrere Dutzend Checkpoints bewertet werden, enthält der so gewählte Wert einen Selektionsoptimismus gegenüber der Validierung, nicht jedoch gegenüber A und B.

Das Holdout-Kriterium ist erfüllt, das Testset-Kriterium verfehlt: B erreicht $66,9\% \geq 60\%$, A bleibt mit $65,7\%$ um 4,3 Prozentpunkte unter der 70 %-Schwelle. Deterministisch werden beide Kriterien verfehlt ($29,1\% / 32,6\%$); zur Einordnung dieses Gaps siehe Abschnitt 7.4. Abbildung 8 zeigt die zugrunde liegenden Trainingsverläufe aller sieben Läufe:

Die hohe Volatilität der Lerndynamik innerhalb der Phasen ist dieselbe Streuungsquelle, die sich in den ± 12 Punkten der Endergebnisse niederschlägt.

v12: Success Rate auf den Validierungs-Seeds, je Curriculum-Phase

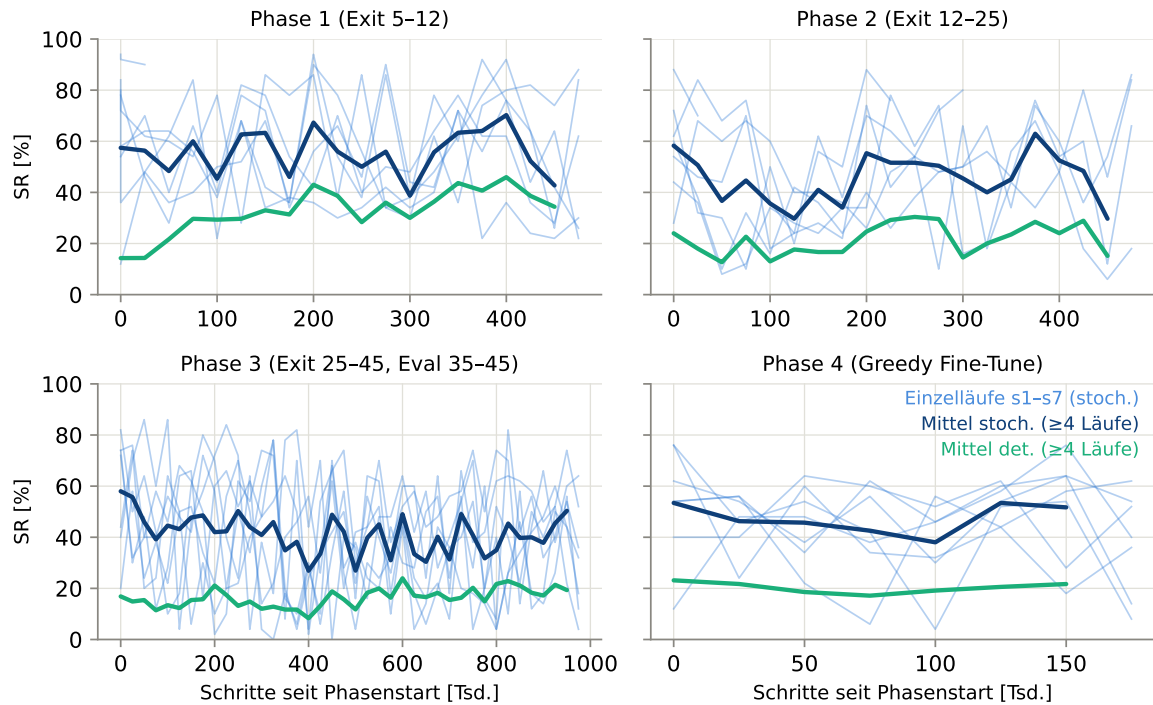


Abbildung 8: Trainingsverläufe der sieben v12-Läufe auf den Validierungs-Seeds 6000–6049, getrennt nach Curriculum-Phase. Dünne Linien: stochastische Erfolgsquote der Einzelläufe; dicke Linien: Ensemble-Mittel stochastisch (dunkelblau) und deterministisch (grün). Da die Schwierigkeit je Phase verschieden ist, sind die Werte nur innerhalb eines Panels vergleichbar.

Der Weg von $n = 3$ zu $n = 7$ ist selbst ein Ergebnis. Eine Zwischenauswertung über die ersten drei Läufe hatte $73,3 \pm 8,3$ (A) und $80,0 \pm 8,0$ (B) ergeben und beide Kriterien scheinbar erfüllt. (Skriptkonsistent gerechnet, d. h. mit demselben Eval-Skript wie die $n=7$ -Auswertung, lauten dieselben drei Läufe 74,0 % bzw. 77,3 %; der Effekt der Aufstockung ist also $74,0 \rightarrow 65,7$ und nicht ein Artefakt des Skriptwechsels.) Da der Abstand zur Schwelle (3,3 Punkte) dort kleiner war als die Streuung und das damalige Bootstrap-Intervall auf A ($[66,7; 80,0]$) die 70 %-Schwelle einschloss, war die Aussage „robust über 70 %“ nicht belegbar. Vier zusätzliche Läufe wurden gezielt zur Klärung dieser Frage gestartet und senkten beide Mittelwerte deutlich. Der Dreier-Mittelwert war zu optimistisch.

Das ist exakt der Effekt, den Henderson et al. [28] für Deep RL beschreiben: Die Varianz über Trainingsläufe ist so groß, dass wenige Seeds systematisch zu Fehlschlüssen führen, weshalb sie Mittelwert $\pm$ Standardabweichung über mehrere Läufe fordern und ausdrücklich vor *best-of-runs*-Berichten warnen. Die vorliegende Arbeit reproduziert diesen Befund am eigenen Gegenstand.

Drei Alternativerklärungen wurden geprüft und ausgeschlossen. Da die vier Zusatzläufe im Mittel schwächer ausfielen (59,5 % gegenüber 74,0 % auf A), wurde geprüft, ob eine systematische Ursache statt Seed-Varianz vorliegt:

1. **Messfehler:** ausgeschlossen. Das für $n = 7$ neu implementierte Evaluationsskript re-

produziert alle sechs deterministischen Werte der Läufe 1–3 exakt (A: 38/26/32; B: 46/44/36); die stochastischen Werte weichen um 2–8 Punkte ab, was dem erwarteten Sampling-Rauschen entspricht.

2. **Code-Stand:** ausgeschlossen. Die Läufe 1–3 und 4–7 entstanden auf unterschiedlichen Commits. Ein Diff über den gesamten Simulationspfad (`src/core`, `src/python`, `src/include`, `python/`, `game_config.json`, `train_curriculum.py`) ergibt genau eine geänderte Datei: `doc_logger.py`, und zwar jene 22 Zeilen, die den Commit-Hash in die `config.json` schreiben. Simulation, Environment und Trainingskript sind identisch; die übrigen Änderungen betreffen ausschließlich Client-Dateien, die nicht in `stoneforge_sim.so` einfließen.
3. **Laufzeitumgebung:** ausgeschlossen. Die installierten Versionen von `torch` und `sb3-contrib` sind älter als beide Lauf-Chargen; die Versions-Pinnung erfolgte deklarativ ohne Neuinstallation.

Ein Welch-Test zwischen beiden Chargen (74,0 vs. 59,5) ist mit $p = 0,149$ nicht signifikant; bei $n = 3$ bzw. $n = 4$ ist die Stichprobe für eine solche Aussage zu klein. Es handelt sich um Seed-Varianz; die $n = 7$ -Werte sind belastbar.

Weitere Beobachtungen. Der bei $n = 3$ auffällige Abstand $B > A$ verschwindet bei $n = 7$ nahezu (66,9 vs. 65,7); beide Sets stammen aus demselben Generator und derselben Distanzverteilung, die frühere Differenz war eine Eigenschaft der kleinen Stichprobe. Ein Gegenteil mit den Phase-2-Zwischenmodellen ergab keine Verbesserung; das finale Modell bleibt das Ausgabemodell.

7.3 Ungelernte Referenzpolitiken und die Grenzen der Erfolgsquote

Eine Erfolgsquote ist ohne Untergrenze nicht interpretierbar: Ohne Bezugspunkt bleibt offen, ob 65 % eine gelernte Fähigkeit ausdrücken oder bereits durch Zufall erreichbar sind.

Verglichen werden zwei ungelernte Politiken, die *dieselbe* Beobachtung erhalten wie der Agent:

- **Random:** gleichverteilte Bewegung; absolute Untergrenze.
- **Kompass-Zufallslauf:** mit Wahrscheinlichkeit ε gleichverteilt zufällig, sonst ein Schritt in die dominante Richtung des Luftlinien-Kompasses ($exitDx/exitDy$). Diese Politik nutzt *zwei* der 229 Merkmale; das lokale 15×15 -Grid wird vollständig ignoriert. Sie hat kein Gedächtnis, keine Parameter und wurde nicht trainiert. Der einzige freie Wert ε wurde ausschließlich auf den Validierungs-Seeds 6000–6049 gewählt, also nach demselben Protokoll wie die Modellselektion; Testset A und Holdout B wurden dafür nicht verwendet.

Als zweite Metrik dient die **Pfadeffizienz** $\eta = d_{\text{BFS}}(s_0) / T_{\text{Erfolg}}$, gemittelt über die erfolgreichen Episoden: das Verhältnis aus kürzestem Weg und tatsächlich benötigten Schritten. $\eta = 1$ bedeutet optimale Navigation, $\eta = 0,05$ einen zwanzigfachen Umweg. Sie ist erforderlich, weil die Erfolgsquote bei einem Episodenlimit von 4000 Schritten und einer mittleren Optimallänge von 40 Schritten (Faktor 100) kaum diskriminiert. Sie greift zugleich die zweite, im Exposé neben der Erfolgsquote genannte Zielgröße auf (Minimierung der *Steps per Episode*), die dort nicht quantifiziert worden war. Tabelle 10 stellt beide Metriken für alle Politiken gegenüber.

Politik	Testset A		Holdout B	
	SR	η	SR	η
Random (gleichverteilt)	$5,2 \pm 2,7$	0,021	$10,4 \pm 1,7$	0,025
Kompass-Zufallslauf, $\varepsilon = 0,4$	$60,4 \pm 4,8$	0,153	$61,6 \pm 5,0$	0,161
Kompass-Zufallslauf, $\varepsilon = 0,9$	92,0 $\pm$ 5,1	0,047	91,6 $\pm$ 4,3	0,048
RecurrentPPO v12 ($n = 7$, stoch.)	$65,7 \pm 12,4$	0,049	$66,9 \pm 12,8$	0,057

Tabelle 10: Ungelernte Referenzpolitiken gegenüber dem trainierten Agenten; Protokoll identisch (je 50 Seeds, Exit 35–45, Limit 4 000). η bezeichnet die Pfadefizienz, die beiden ε -Werte die auf der Validierung erfolgs- bzw. effizienzoptimale Einstellung. Alle Baseline-Werte sind Mittel über fünf Wiederholungen mit unterschiedlichem Politik-Zufallsstrom (RNG-Seeds 100–104) bei festen Weltseeds. Erzeugt mit `scripts/eval_baselines.py`. Die Random-Zeile reagiert bei ihrem niedrigen Niveau merklich auf die Wahl des Politik-Zufallsstroms (mit den RNG-Seeds 0–4: $10,4 \pm 4,6$ auf A); die Kompass-Zeilen sind stabil ($89,6 \pm 1,7$ bei $\varepsilon = 0,9$).

Befund. Der Kompass-Zufallslauf übertrifft den trainierten Agenten auf *beiden* Achsen. Bei vergleichbarer Pfadefizienz ($\eta \approx 0,05$) erreicht er 92 % statt 65,7 %; bei vergleichbarer Erfolgsquote (≈ 60 %) ist er mit $\eta = 0,15$ rund dreimal wegeffizienter. Der Agent liegt zwar deutlich über der Zufallsuntergrenze von 5–10 %, erreicht aber an keinem Betriebspunkt die Leistung einer Heuristik aus vier Zeilen Code, die 227 seiner 229 Eingangsmerkmale ignoriert.

Kein Episodenbudget kehrt das Bild um. Naheliegend wäre der Einwand, das Problem liege allein am großzügigen Limit von 4 000 Schritten. Das lässt sich exakt prüfen: Wer eine Welt bei Schritt T löst, löst sie unter jedem Budget $\geq T$; aus einem einzigen Lauf mit protokollierten Erfolgsschritten folgt daher die vollständige Kurve *Erfolgsquote über Budget* (Tabelle 11). Das Ergebnis ist eindeutig: Bei *jedem* Budget erreicht oder übertrifft mindestens eine der beiden ungelernten Politiken alle drei trainierten Läufe. Ein engeres Episodenlimit ist als Korrektur damit nicht ausreichend.

Budget	Kompass-Zufallslauf		RecurrentPPO v12		
	$\varepsilon = 0,4$	$\varepsilon = 0,9$	Lauf 1	Lauf 2	Lauf 3
120 (3× Optimum)	4 %	0 %	0 %	2 %	0 %
200	22 %	0 %	0 %	4 %	0 %
300	28 %	0 %	2 %	10 %	6 %
500	36 %	10 %	12 %	22 %	12 %
800	54 %	18 %	20 %	32 %	30 %
1 500	62 %	54 %	40 %	48 %	54 %
4 000	64 %	88 %	64 %	78 %	74 %

Tabelle 11: Erfolgsquote auf Testset A in Abhängigkeit des Episodenbudgets (BFS-Optimum $\varnothing$ 40 Schritte). Hervorgehoben ist je Zeile der höchste Wert.

Was daraus folgt und was nicht. Der Befund entwertet weder die Plattform noch die methodische Arbeit dieses Projekts; er entwertet die *Aufgabenstellung* in ihrer aktuellen Parametrisierung. Drei Eigenschaften wirken zusammen: (1) Das Episodenlimit übersteigt die Optimallänge um den Faktor 100, sodass ungerichtete Suche genug Zeit hat; (2) der Luftlinien-Kompass liefert dauerhaft eine grob richtige Richtung, sodass ein verrauschter Gradientenaufstieg genügt; (3) die erzeugten Welten sind hinreichend offen, dass Sackgassen durch Zufallsbewegung wieder verlassen werden. Unter diesen Bedingungen ist Gedächtnis für die

Erfolgsquote schlicht nicht erforderlich, und die Metrik kann folglich nicht messen, ob es gelernt wurde.

Damit ist die zentrale Leistungsaussage dieser Arbeit einzuschränken: Belegt ist, dass der Agent die Aufgabe löst und dabei deutlich über der Zufallsuntergrenze liegt; *nicht* belegt ist, dass er eine Strategie gelernt hat, die über gerichtete Zufallssuche hinausgeht. Diese Einschränkung betrifft auch die Gegenüberstellung in Abschnitt 7.1: Wenn eine kompassgetriebene Zufallspolitik 92 % erreicht, ist der Vergleich „LSTM ohne BFS“ gegen „MLP mit BFS“ kein Test der Gedächtnisfähigkeit, sondern misst überwiegend, wie gut die jeweilige Politik den Kompass ausnutzt und wie stark sie in Schleifen gerät.

Konsequenz für Folgearbeiten. Die naheliegende Reaktion ist nicht, den Agenten weiter zu tunen, sondern die Aufgabe selbst zu korrigieren. Da Tabelle 11 zeigt, dass eine Änderung des Episodenbudgets allein nicht genügt, führt der Weg über die Weltgenerierung: Der Wandanteil muss so weit erhöht werden, dass ungerichtete Suche scheitert, ohne die Lösbarkeit zu gefährden. Die Baseline-Politik dient dabei als Kalibrierinstrument, mit dem sich ein geeigneter Schwierigkeitsgrad *vor* jedem weiteren Training bestimmen lässt. Das Vorgehen ist in Abschnitt 8 ausgeführt. Erst unter einem solchen Regime lässt sich die Forschungsfrage dieser Arbeit beantworten.

Eine unabhängige Wiederholung der Modellmessung mit anderem Zufallsstrom ergab 68,6 % auf Testset A statt der in Tabelle 9 berichteten 65,7 %. Die Abweichung von knapp drei Punkten entspricht dem erwarteten Sampling-Rauschen und ändert die Aussage nicht.

7.4 Der Det/Stoch-Gap: Weglängen-Analyse und POMDP-Einordnung

Der Abstand zwischen stochastischer und deterministischer SR ist kein Schwelleneffekt, sondern wächst kontinuierlich mit der Weglänge (Abbildung 9; 120 Validierungs-Welten, nach Startdistanz gruppiert): Bei 5–15 Feldern erreicht die deterministische Betriebsart 79 %, bei 35–45 Feldern noch 31 %, während die stochastische zwischen 100 % und 82 % bleibt. Die Interpretation ist kumulativ: Je länger der Weg ohne direkte Zielsicht (Sichtradius 7), desto mehr Kreuzungsentscheidungen unter unsicherem Belief-State; an jeder liegen die Aktionswahrscheinlichkeiten nahe beieinander, und jede einzelne kann die Argmax-Politik in eine Oszillationsschleife führen, aus der Sampling entkommt. Der Gap ist damit teils eine Gedächtnis-Limitation, teils dem POMDP-Charakter geschuldet (unter Unsicherheit ist eine stochastische Politik nicht per se suboptimal, Abschnitt 2).

Zur theoretischen Einordnung ist eine Unterscheidung nötig, die in der Zwischendokumentation dieses Projekts zunächst unscharf blieb. Singh et al. zeigen, dass in POMDPs *stochastische gedächtnislose* Politiken jede deterministische gedächtnislose erheblich übertreffen können [6]. Dieses Resultat erklärt den hier beobachteten Gap jedoch *nicht*, denn die verwendete Politik ist nicht gedächtnislos: Über einem hinreichend genauen Belief-State existiert wieder eine optimale deterministische Politik. Dass die Argmax-Auswertung des LSTM-Agenten dennoch deutlich abfällt, ist daher primär ein Hinweis auf einen *unvollständigen* Belief-State, also eine Gedächtnis-Limitation, und nur mittelbar eine Eigenschaft der Aufgabe. Singh et al. liefern die Motivation dafür, warum stochastisches Verhalten unter partieller Beobachtbarkeit überhaupt nützlich sein kann, nicht aber eine Rechtfertigung des konkreten Gaps.

Der Teil der Begründung, der auch *mit* Gedächtnis trägt, stammt aus der Generalisierungs-Anforderung: Ghosh et al. zeigen, dass bereits das Ziel, auf unbekannte Level (hier: neue Seeds) zu generalisieren, selbst eine Form partieller Beobachtbarkeit induziert (*epistemic POMDP*), deren optimale Politik stochastisch ist und die durch mehr Gedächtnis über die *aktuelle* Episode nicht aufgelöst wird [25]. Die Unsicherheit liegt hier über der Identität der Welt, nicht über dem Zustand innerhalb einer bekannten Welt. Auf dieses Argument, nicht auf Singh et al., stützt sich die Wahl der stochastischen Evaluation als Primärmetrik. Der Gap ist damit teils erwartbar (epistemische Unsicherheit über Seeds), teils ein offenes Defizit

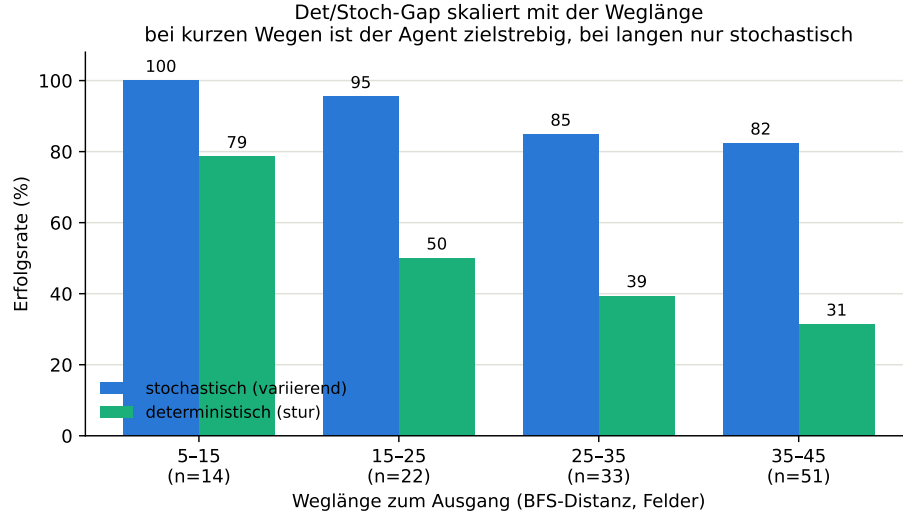


Abbildung 9: Erfolgsquote nach Startdistanz zum Exit (120 Validierungs-Welten, finales v12-Modell, Limit 4000): Die deterministische SR fällt kontinuierlich mit der Weglänge, die stochastische bleibt hoch.

des gelernten Belief-States; die Trennung beider Anteile ist mit den vorliegenden Daten nicht möglich und wird als Diagnoseschritt 1 im Ausblick (Abschnitt 8) vorgeschlagen.

7.5 Versuche zur Schließung des Det/Stoch-Gaps

Um zu prüfen, ob der Gap durch gezielte Eingriffe verkleinerbar ist, wurden drei Experimente durchgeführt, die je einen inhaltlichen Faktor gegenüber der v12-Baseline verändern (Setup in Tabelle 6): **E1** verstärkt den Critic (vf_coef 0,5 $\rightarrow$ 1,0), **E2** glättet den Curriculum-Übergang (Phase 3 nur bis Distanz 35 statt 45), **E3** vergrößert das Gedächtnis (LSTM 256 $\rightarrow$ 512).

Aussagekraft. Diese drei Läufe sind Einzelmessungen ($n = 1$) gegen eine Baseline aus drei Läufen. Die in Abschnitt 7.2 gemessene Streuung über Trainingsläufe beträgt $\sigma \approx 12$ Prozentpunkte; ein Einzellauf kann also allein durch Seed-Varianz um mehr als 20 Punkte von der Baseline abweichen. Die Arme sind daher *kein* Wirksamkeitsnachweis, sondern eine Vorauswahl: Sie können eine Hypothese schwächen, aber nicht widerlegen. Konsequenterweise wird hier nur die schwache Form berichtet.

Kein Arm zeigt einen Hinweis auf Verbesserung. E1 hebt die deterministische SR (42 % statt 32 % auf A), liegt stochastisch aber niedriger (56 % statt 73 %). Beide Differenzen liegen innerhalb bzw. knapp oberhalb von $1,5\sigma$ und sind bei $n = 1$ nicht belastbar; die Richtung (mehr Value-Gewicht $\rightarrow$ determinierteres Verhalten) ist plausibel, aber unbestätigt. E2 liegt mit A 26 % rund 4σ unter der Baseline; das ist auch bei $n = 1$ deutlich und mechanistisch erklärbar, weil das Modell auf den längsten, eval-relevanten Distanzen untertrainiert bleibt. Zu beachten ist, dass E1 und E2 zusätzlich nur 400k statt 1,0 Mio. Phase-3-Schritte erhielten; ihr Rückstand ist also teilweise ein Budget-, kein Faktoreffekt. E3 liegt mit A 44 % rund $2,5\sigma$ unter der Baseline, ist ein voller Lauf mit identischem Budget und zeigt in *allen* Curriculum-Phasen Werte auf oder unter der 256er-Baseline bei rund dreifachem Rechenaufwand. Das ist der belastbarste der drei Befunde: Mehr Gedächtnis-Kapazität bedeutet in dieser Umgebung jedenfalls nicht mehr Gedächtnis-Nutzen. Er passt zu den bekannten Grenzen rekurrenter Politiken in prozeduraler Navigation, deren Gedächtnisanforderungen über reine Kapazität hinausgehen [19]. Eine Replikation von E1 und E3 mit je zwei weiteren Seeds steht aus und ist der nächste offene Punkt (Abschnitt 8).

Ergänzend wurde geprüft, ob der Gap lediglich ein Kalibrierungsartefakt der Argmax-Auswertung ist. Ein Temperatur-Sweep zwischen reinem Argmax ($T=0$) und vollem Sampling ($T=1$) auf den drei v12-Modellen (Tabelle 12) zeigt einen *monotonen* Verlauf: Mehr Stochastik ist durchweg besser, keine Zwischentemperatur übertrifft das volle Sampling. Der Gap ist damit *kein* Argmax-Artefakt, sondern die empirische Signatur des POMDP-Resultats [6]. Bereits $T=0,5$ holt den Großteil des Argmax-Defizits zurück (A: +31, B: +29 Prozentpunkte); dies ist ein geeigneter Betriebspunkt, falls ein weniger zufälliges Verhalten gewünscht ist.

Temperatur	Testset A	Holdout B
Argmax ($T=0$)	$32,0 \pm 4,9$	$42,0 \pm 4,3$
$T = 0,25$	$52,0 \pm 0,0$	$52,7 \pm 1,9$
$T = 0,5$	$63,3 \pm 4,1$	$71,3 \pm 6,6$
$T = 0,75$	$63,3 \pm 7,5$	$72,0 \pm 6,5$
Sampling ($T=1$)	$70,7 \pm 8,2$	$79,3 \pm 6,6$

Tabelle 12: Erfolgsquote in Abhängigkeit der Sampling-Temperatur (Mittel $\pm$ Std über die drei v12-Läufe, je 50 Seeds, Limit 4000).

Ein Hinweis zur Konsistenz: Die Zeile $T=1$ (70,7% auf A) misst dieselbe Größe wie die oben genannten 73,3% und die in Abschnitt 7.2 nachgerechneten 74,0%. Die drei Werte stammen aus unabhängigen Messungen mit drei Skripten; die Streuung von rund drei Punkten entspricht dem Sampling-Rauschen über 50 Bernoulli-Versuche. Das ist kein Widerspruch, zeigt aber, dass Unterschiede dieser Größenordnung nicht interpretierbar sind. Vergleiche *innerhalb* der Tabelle bleiben zulässig, da sie aus demselben Lauf stammen.

Zusammengefasst widersteht der Det/Stoch-Gap dem Tuning einzelner Stellschrauben; er ist teils fundamental (POMDP). Nicht-architektonische Hebel wie Hilfs- bzw. Repräsentationsverluste oder gezielt mehr Langdistanz-Training bleiben ergänzend denkbar; das *harte*, fundamentale Langhorizont-Belief-Tracking ist jedoch am ehesten über strukturiertes statt größeres Gedächtnis anzugehen (Abschnitt 8).

8 Fazit und Ausblick

8.1 Ergebnis

Über sieben unabhängige Läufe erreicht der Agent $65,7\% \pm 12,4$ (Testset A) und $66,9\% \pm 12,8$ (Holdout B) stochastische Erfolgsquote. Das Holdout-Kriterium ($\geq 60\%$) ist damit erfüllt, das Testset-Kriterium ($\geq 70\%$) verfehlt.

Diese Zahlen sind jedoch nur gegen die Zufallsuntergrenze von 5–10% aussagekräftig, nicht gegen eine kompetente ungelernte Referenz: Ein Kompass-Zufallslauf erreicht 92% und ist bei vergleichbarer Erfolgsquote zugleich dreimal wegeffizienter (Abschnitt 7.3). Bei einem Episodenbudget vom Hundertfachen der Optimallänge misst die gewählte Metrik nicht die Fähigkeit, um die es geht, und ein engeres Budget behebt das nicht (Tabelle 11). **Die Forschungsfrage ist mit dem vorliegenden Aufbau daher nicht entscheidbar: Die Umgebung erfordert das Gedächtnis nicht, dessen Nutzen sie messen soll.** Dieser Befund betrifft die Interpretation sämtlicher Leistungszahlen der Arbeit, einschließlich der Gegenüberstellung in Abschnitt 7.1.

8.2 Beiträge

Methodisch. Die Zwischenauswertung über drei Läufe hatte 73,3% / 80,0% ergeben und beide Kriterien scheinbar erfüllt. Die Aufstockung auf sieben Läufe senkte beide Werte deutlich. Ein auf drei Läufen berichtetes „beide Ziele erfüllt“ wäre nicht falsch gemessen, aber

nicht haltbar gewesen. Zusammen mit dem Baseline-Befund reproduziert die Arbeit damit zweimal am eigenen Gegenstand, wovor Henderson et al. [28] warnen: zu wenige Läufe und ein fehlender Referenzpunkt führen unabhängig voneinander zu Fehlschlüssen.

Inhaltlich. Ein rekurrenter Agent ohne BFS-Gradientenfeatures erreicht mindestens dieselbe stochastische Leistung wie ein MLP *mit* diesen Features; der LSTM-Zustand ersetzt die vorberechnete lokale Pfadinformation. Diese Aussage ist doppelt begrenzt: durch die Einschränkungen der Gegenüberstellung (Tabelle 8) und dadurch, dass der globale Luftlinien-Kompass in beiden Bedingungen erhalten bleibt. Ersetzt wird also die *lokale* Wegweiserinformation, nicht jede globale Zielinformation.

Technisch. Entstanden ist eine vollständige Experimentierplattform: ein C++-Kern mit deterministischer prozeduraler Weltgenerierung, über `pybind11` als Gymnasium-Umgebung gekapselt, dazu Curriculum-Trainingspipeline, standardisierte Evaluationsskripte, Live-Monitoring und ein spielbarer Client mit KI-Zuschaumodus (Abschnitt 6). Die deterministische Reproduzierbarkeit pro Seed und die durchgängige Experiment-Protokollierung waren die Voraussetzung für die Root-Cause-Analysen im Anhang 10 und sind für Folgearbeiten wiederverwendbar.

8.3 Einschränkungen

1. **Die Erfolgsmetrik ist nicht trennscharf** und wird von einer ungelerten Heuristik übertroffen (Abschnitt 7.3). Dies ist die schwerwiegendste Einschränkung, weil sie alle übrigen Leistungsaussagen relativiert.
2. **Das Testset-Kriterium wird verfehlt** (65,7 % statt ≥ 70 %); nur das Holdout-Kriterium ist erfüllt.
3. **Der Det/Stoch-Gap bleibt bestehen** (29,1 % bzw. 32,6 % deterministisch). Er wächst mit der Weglänge und ist monoton in der Sampling-Temperatur, also kein Argmax-Artefakt. Theoretisch erwartbar ist nur der Anteil aus der epistemischen Unsicherheit über den Seed [25]; der Rest ist ein Defizit des gelernten Belief-States. Die drei Versuche zu seiner Schließung sind Einzelläufe und belegen nicht, dass er tuning-resistent *ist*.
4. **Begrenzte statistische Präzision:** Die Streuung über Läufe beträgt rund ± 12 Punkte; das 95 %-Konfidenzintervall auf Holdout B reicht bis 55,0 % hinab, also unter die im Mittel erfüllte Schwelle.
5. **Ein einziger Generator und keine quantifizierte Baseline nicht-rekurrenter Verfahren** (DQN, A2C) auf der finalen Umgebung: Der frühe DQN-Anlauf scheiterte vor der v11-Revision und wurde nicht wiederholt.

8.4 Ausblick

Die Reihenfolge der folgenden Punkte ergibt sich aus dem Befund von Abschnitt 7.3: Solange die Aufgabe die untersuchte Fähigkeit nicht erfordert, ist jede weitere Optimierung des Agenten nicht interpretierbar. Die Reparatur des Messregimes steht deshalb vor allem anderen.

1. **Härtung der Umgebung.** Der Wandanteil wird derzeit pro Biom fest vorgegeben (7 % im Wald bis 25 % im Bergland) und durch einen zellulären Automaten geglättet. Er sollte zu einem steuerbaren Parameter werden und so weit erhöht werden, bis der Kompass-Zufallslauf einbricht, die Lösbarkeit aber bei 100 % bleibt. Das Vorgehen ist

zweistufig: erst die Baseline-Politik als Kalibrierinstrument einsetzen, dann trainieren. Ein Trainingslauf ohne diese Vorabprüfung wäre erneut nicht interpretierbar.

2. **Billige Diagnosen vor jedem Architekturwechsel.** (a) Eine lineare Probe auf dem LSTM-Hidden-State, die Exit-Richtung bzw. BFS-Distanz vorhersagt: Kollabiert sie auf langen Wegen, liegt ein Belief-Tracking-Limit vor, sonst liegt das Problem in der Politik. (b) Entropie-Annealing in Phase 4 mit Modellselektion auf der deterministischen Erfolgsquote. (c) One-Hot- oder Embedding-Kodierung des lokalen Grids statt der bisherigen ordinalen Tile-IDs; dass das Grid kaum genutzt wird, legt die Baseline-Analyse nahe. (d) Replikation von E1 und E3 mit je zwei weiteren Seeds.
3. **Restpotenzial in RecurrentPPO.** Ein Auxiliary-Loss-Kopf, der aus dem Hidden-State die BFS-Distanz vorhersagt, formt den Belief-State explizit und nutzt das Orakel als Trainings-, nicht als Testzeitsignal. Ergänzend Burn-in für Sequenzstarts nach R2D2 [18], temperaturkalibrierte Evaluation ($T = 0,5$ ohne Neutraining) sowie vollwertiges Prioritized Level Replay [22] und Self-Imitation [29].
4. **Architekturvergleich als Folgearbeit.** Der Wechsel auf Transformer-basierte In-Context-Verfahren (z. B. AMAGO [21]) ist die meistgenannte Richtung, hier aber bewusst nur eine Hypothese. Denn erstens ist nicht belegt, dass der Gap ein Kapazitätsproblem ist: Eine unter hoher Entropie trainierte Politik kann ihr Restrauschen zum Verlassen von Schleifen nutzen, ohne je einen Anreiz für eine deterministisch tragfähige Aktionswahl gehabt zu haben. Zweitens bleiben rekurrente Modelle im breitesten verfügbaren POMDP-Vergleich gegenüber den dort geprüften Attention-Modellen konkurrenzfähig; zu beachten ist, dass es sich dabei um *lineare* Transformer-Varianten handelt und nicht um volle Softmax-Attention, wie AMAGO sie verwendet [20]. Die eigene E3-Ablation zeigte, dass mehr Gedächtniskapazität hier schadet. Erst wenn Schritt 2 ein Kapazitätslimit nachweist, wird der Wechsel zur begründeten nächsten Frage; dann als kontrollierter Vergleich auf demselben Generator und Protokoll.

9 Anhang: Reproduzierbarkeit

Dieser Anhang folgt den Reproduzierbarkeits-Checklisten von Pineau et al. [27] und Henderson et al. [28]. Alle Trainingsläufe sind über Seed, gepinnte Umgebung und `config.json` reproduzierbar; die vollständige Änderungshistorie inklusive verworfener Varianten liegt im Experiment-Changelog. Tabelle 13 listet die verwendeten Software-Versionen.

Komponente	Version / Wert
Python	3.12.13
Betriebssystem	macOS (arm64)
Rechen-Device	CPU (bewusst gewählt, s. Performance-Analyse)
PyTorch	2.12.0
Stable-Baselines3 / sb3-contrib	2.8.0 / 2.8.0
Gymnasium	1.2.3
NumPy	2.4.4
Abhängigkeiten (gepinnt)	<code>requirements.txt</code> (==) + <code>requirements.lock.txt</code>

Tabelle 13: Systemumgebung. Nach jeder Änderung an `game_config.json` oder am C++-Kern ist ein Rebuild des Python-Bindings erforderlich.

Reproduzierbarkeits-Checkliste:

- **Code & Konfiguration:** Repository mit Trainings-/Eval-Skripten; je Lauf eine `config.json` mit allen Hyperparametern (Tab. 5, 6).

- **Durchsuchter Hyperparameter-Bereich:** vollständig im `CHANGELOG.md` dokumentiert, inklusive *negativer* Ergebnisse (v7–v10-Fehleranalyse, Tab. 14; Iterationen E1–E3).
- **Anzahl Läufe:** sieben unabhängige Seeds; berichtet werden Mittelwert $\pm$ Standardabweichung, *kein* Best-of-Runs (Kernkritik von Henderson et al. [28]).
- **Datentrennung:** disjunkte Seed-Mengen: Validierung (6000–6049) für Phasenwechsel und Modellauswahl, Testset A (7000–7049) und Holdout B (8000–8049) ausschließlich für die finale Messung.
- **Evaluationsprotokoll:** 50 Seeds, Episodenlimit 4 000 (= Umgebungs-Maximum), stochastisch *und* deterministisch. Bei der rekurrenten Politik werden LSTM-Zustand und Episoden-Start-Signal korrekt an `predict()` durchgereicht und je Episode zurückgesetzt; ein häufiger Fehler beim Evaluieren rekurrenter Politiken wird damit vermieden.
- **Statistische Unsicherheit:** Bei 50 Seeds beträgt das 95 %-Konfidenzintervall einer Erfolgsquote bis zu ± 14 Prozentpunkte; Unterschiede unterhalb dieser Größe sind nicht signifikant zu interpretieren.
- **Rechenaufwand:** je voller Curriculum-Lauf ca. 8 h (CPU) bei voller Taktung; die Läufe 4–7 liefen durch Energie-Drosselung (zugeklappter Laptop) 29–36 h Wall-Clock, ohne Ergebnisrelevanz, da das Training step-basiert und das Curriculum leistungsbasiert ist. Trainingszeiten je Lauf in `results.json`.
- **Zufallsquellen:** Seed steuert Umgebungs-Generierung und -Reset (deterministisch); die stochastische Politik samplet Aktionen (torch-RNG). Restnicht-Determinismus (Thread-Scheduling) wird durch die Mittelung über sieben Läufe aufgefangen.

10 Anhang: Entwicklungsverlauf von der Baseline zum LSTM-Agenten

Primärquelle der folgenden Chronologie ist das durchgängig geführte Experiment-Changelog (`CHANGELOG.md`), in dem jede Änderung mit Problem, Lösung und Messergebnis festgehalten wurde. Negative Ergebnisse sind bewusst enthalten; sie haben die Architekturentscheidungen maßgeblich geprägt.

10.1 Etappe 1 (Mai 2026): Baselines mit BFS-Orakel

Nach Fertigstellung der Spiel-Engine (Mitte Mai) begann das RL-Training mit Features aus dem BFS-Distanzfeld *in der Observation* (5 Werte: aktuelle Distanz + 4 Nachbarrichtungen), einem „Orakel“, das dem Agenten die lokal richtige Richtung lieferte.

Erster Anlauf mit DQN: Der erste Agent (DQN) lernte nicht. Diagnose: korrupter Replay-Buffer, 2-Zellen-Oszillationen, und eine Visited-Mask in der Observation (455 Dimensionen), die sich jeden Schritt änderte und den Q-Wert-Argmax destabilisierte. Konsequenz: Wechsel auf PPO, Visited-Mask entfernt.

BFS-Kraftfeld-Bug: Ein Oracle-Test (greedy dem BFS-Feld folgen) scheiterte in 3/3 Seeds; `bfsDistanceAt()` lieferte für Wand-Tiles einen Manhattan-Fallback, wodurch Wände als attraktive Richtung erschienen. Nach dem Fix (Wand $\rightarrow$ 9999): Oracle 3/3. Dieser Test wurde zum Standard-Werkzeug, um Umgebung und Reward von Trainingsproblemen zu trennen.

Curriculum-Lektion: Phase-1-Training (Exit 5–12) erreichte 56 %, aber der naive Transfer auf Phase 2 (12–25) kollabierte (22 % $\rightarrow$ 6 %); die zuvor gelernte Kurzdistanz-Politik ließ

sich nicht überschreiben. Lösung: Mixed-Distribution-Training (Exit 5–45). Damit erreichte *Phase 3* 86 % stochastisch auf Testset A (drei unabhängige Läufe: $85,3 \pm 3,8$ % auf A, $79,3 \pm 8,1$ % auf B) und *Phase 4* nach Reward-Tuning 98 % (A) / 94 % (B).

Der deterministische Kollaps: Trotz 98 % stochastischer SR lag die Argmax-Auswertung bei 2 %. Die Analyse fand die Wurzel in der Feature-Kodierung: Die BFS-Absolutwerte (/64) unterschieden sich zwischen Nachbarzellen nur um $\approx 0,016$, zu wenig für eine robuste Richtungspräferenz, und Wände waren durch Clamping nicht von „weit weg“ unterscheidbar (Aliasing). Der Wechsel auf *Delta-Kodierung* (Richtungssignal $\pm 0,5$ statt 0,016, Wand = +1,0) hob die deterministische SR auf 42 % (offene Welt) bzw. 100 % (wanddichte Hard-World); Temperature-Sampling ($\tau = 0,2$) erreichte 100 % bei $\varnothing$ 90 Schritten. Ein Experiment, exitDx/exitDy zu entfernen („Phase 5“), scheiterte (max. 40 %); der Luftlinien-Kompass ist als Fernbereichssignal notwendig.

10.2 Etappe 2 (Juni 2026): Kernbeitrag, Navigation ohne Orakel

Mit funktionierender Baseline wurde die Forschungsfrage geschärft: *Kann der Agent auch ohne BFS-Orakel in der Observation navigieren?* Der Wechsel auf RecurrentPPO (LSTM) mit Curriculum, Explorations-Bonus und step_frac-Feature (231-dim Obs, kein BFS) wurde am 07.06.2026 gestartet und lieferte nach der Auswertung am Folgetag das zentrale Ergebnis: **86 % stochastisch / 36 % deterministisch** auf Testset A und 68 % / 18 % auf Holdout B; damals waren beide Zielkriterien stochastisch erfüllt (der final über sieben unabhängige Läufe reproduzierte Wert liegt bei 65,7%/66,9 % für A/B, Abschnitt 7.2), erstmals ohne Wegweiser-Orakel (Abbildung 10, Ablation in Tabelle 8). Parallel wurde die POMDP-These formuliert (Abschnitt 7.1) und, inspiriert vom PokéRL-Projekt, Infrastruktur ergänzt: Seed-Pool (Swarm), WebSocket-Live-Map, Heatmap-Evaluation, Early-Stop-Truncation.

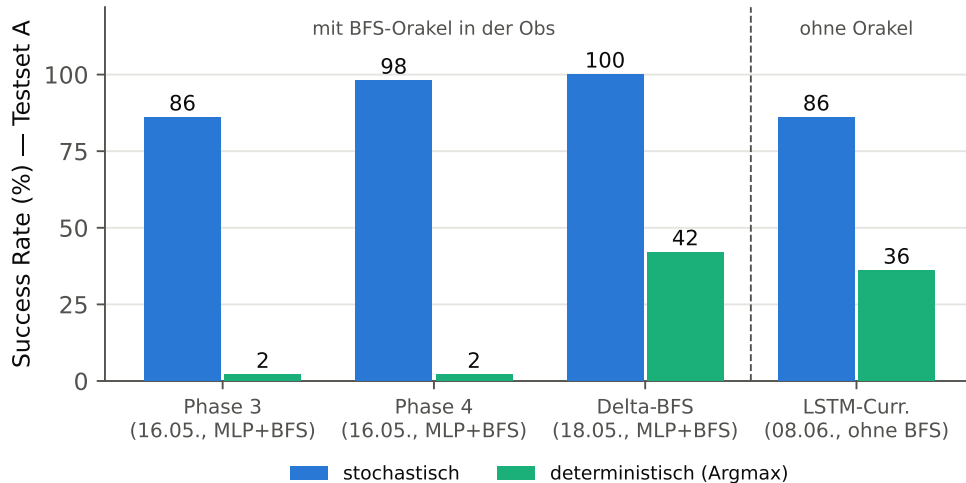


Abbildung 10: Entwicklung über die Modellgenerationen (Testset A, Exit 35–45). Links die MLP-Baselines mit BFS-Orakel in der Observation, rechts der LSTM-Agent ohne Orakel.

Methodische Härting (11.06.): Ein Data-Leakage-Fehler wurde entdeckt und behoben. Der Trainings-Callback hatte Modellauswahl und Phasenwechsel auf den *Test-Seeds* (7000er) gesteuert; seitdem existiert das getrennte Validierungsset (6000er). Zusätzlich wurde die Lösbarkeit aller relevanten Welten bewiesen (3 150 Seeds, 100 %).

10.3 Etappe 3 (13.–25.06.): Fehlkonfigurierte Hyperparameter und Root-Cause-Analyse

Symptom: Eine Serie von Verbesserungsversuchen (v6–v9: Wand-Penalty-Entfernung, visit_count- und Action-Buffer-Features, diverse Batch-/Epochen-Kombinationen) scheiterte durchgängig (SR 0–12 %, `approx_k1` ≈ 0). **Root-Cause:** Die Aufklärung gelang durch *Reverse-Engineering* des letzten funktionierenden Modells (v2): Die zwischenzeitlichen „Optimierungen“ hatten unbemerkt die kritischen Hyperparameter verstellt (Tabelle 14); dominanter Faktor war `ent_coef` (0,05 $\rightarrow$ 0,01: der Agent explorierte nicht mehr genug, um den Exit je zu finden). **Ergebnis:** Die v2-Reproduktion (v10) stellte das Lernen wieder her; der abgeschlossene Lauf vom 25.06. erreichte 86 % (Validierung) nach 2,2 Mio. Steps in 3 h 12 m; das aktuelle Referenzmodell (Abbildung 7).

Parameter	v2 (86 %)	v7–v9 (0–12 %)	Wirkung des Fehlers
<code>ent_coef</code>	0.05	0.01	zu wenig Exploration; Exit wird nie gefunden
<code>n_steps</code>	256	512	BPTT zu lang $\rightarrow$ verschwindende Gradienten
<code>batch_size</code>	8	256	zu wenige Gradient-Updates pro Rollout
Obs-Dimension	231	249	Zusatzfeatures ohne funktionierende Basis

Tabelle 14: Root-Cause-Analyse: `ent_coef` erwies sich als dominanter Faktor. Die Reproduktion der v2-Konfiguration (v10) stellte das Lernen wieder her.

10.4 Etappe 4 (06.07.2026): Umgebungsrevision v11 und Performance

Ein systematisches Audit (Umgebung, Reward, Trainingspipeline, Usability) führte zu zehn dokumentierten Änderungen; Details in Abschnitt 10.7 und Abschnitt 10.8. Höhepunkte: präzise Schwierigkeitssteuerung über BFS-Laufweg, Behebung eines prozess-globalen Config-Lecks (das zuvor unbemerkt die Phase-3-Trainingsverteilung verschob), Entfernung toter Features und Straf-Terme, sowie die Wahl der CPU als Rechen-Device. Die an diesem Tag ebenfalls vorgenommene Verdopplung der Trainingsgeschwindigkeit über `batch_size` 8 $\rightarrow$ 64 galt zunächst als validiert, wurde am Folgetag jedoch widerlegt (Abschnitt 10.6) und zurückgenommen.

10.5 Etappe 5 (07.07.2026): Erster v11-Gesamtlauf, Monitoring-Redesign und Seed-Replay-Analyse

Erster v11-Gesamtlauf (Seed 0, laufend). Der erste vollständige Curriculum-Lauf auf der revidierten v11-Umgebung (`models/ppo_lstm_curriculum_v11`) zeigt folgendes Zwischenbild (Abbildung 11): Phase 1 endete am Step-Limit bei 42 % (stochastisch), deutlich langsamer als die Batch-Validierung derselben Konfiguration (52 % bereits bei 150 k). Die Diagnose im Trainingslog: Die Policy-Updates sind durchgehend gesund (`approx_k1` ≈ 0.03 , stabile Entropie), aber die `explained_variance` des Critics verharrt bei ≈ 0.1 (Validierungslauf: 0.72); der Lauf wurde zunächst als Kandidat für eine ungünstige Zufallsinitialisierung eingestuft. (Nachtrag: Die systematische Fehlersuche am selben Abend, Abschnitt 10.6, fand die tatsächliche Ursache: die Batch-Größe.) Bemerkenswert: Direkt nach dem Wechsel in Phase 2 stieg die SR auf 48 % bei größerer Exit-Distanz; die breitere Aufgabenverteilung wirkt als Regularisierer.

Live-Monitoring-Redesign. Das WebSocket-Dashboard aus Etappe 2 wurde nach Dashboard- und Farb-Literatur überarbeitet (Abbildung 12): (a) Charts auf uPlot mit EMA-Glättung nach TensorBoard-Vorbild; (b) die SR-Kurve zeigt det. und stoch. getrennt; der zentrale Untersuchungsgegenstand war zuvor im Monitoring unsichtbar; (c) die Explorations-Heatmap

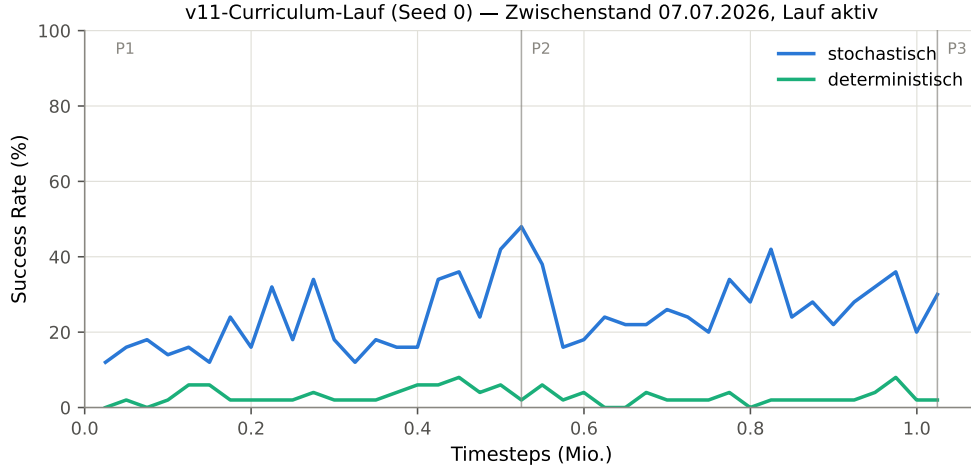


Abbildung 11: Lernkurve des ersten v11-Gesamtlaufs (Seed 0, Zwischenstand in Phase 3), beide Betriebsarten. Vertikale Linien: Phasenwechsel; die x-Achse ist über die Phasen kumuliert. Erzeugt mit `scripts/plot_run_curve.py`.

wechselte von der Jet- auf die perzeptuell uniforme Viridis-Skala (Jet erzeugt künstliche Kontraste und ist nicht farbfeldsichten-tauglich); (d) die Agenten-Minimaps rekonstruieren aus dem gestreamten 15×15 -Sichtfeld eine wachsende Weltkarte (Wände, Bäume, Exit) statt nur der Agentenposition. Beim Umbau wurden zwei Fehler gefunden und behoben: Der Exit-Richtungspfeil war seit der Obs-Umstellung auf 229 Dimensionen stumm defekt, und der SB3-Stepzähler springt beim Phasenwechsel zurück (es wird das Bestmodell der Vorphase geladen), was unkorrigiert alle stepbasierten Zeitachsen verfälscht.

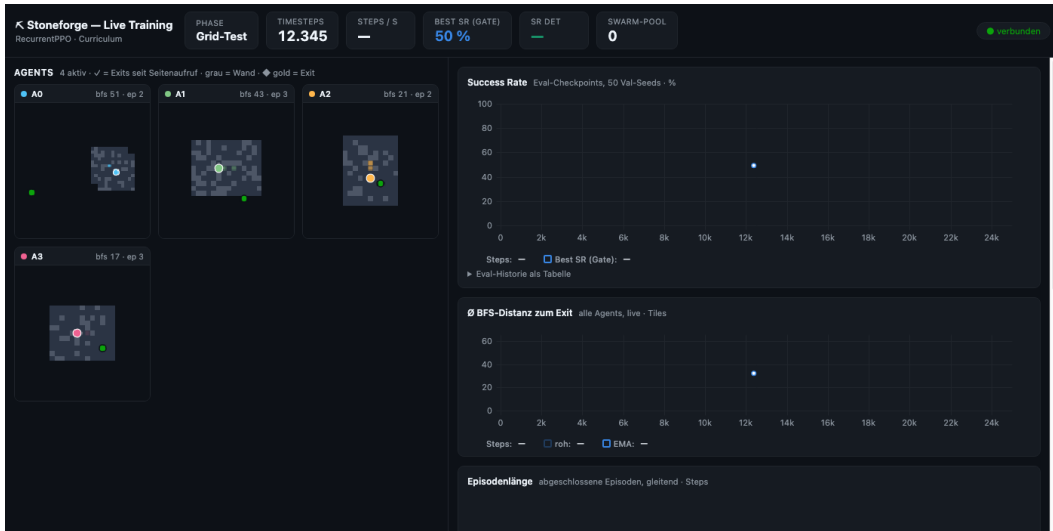


Abbildung 12: Live-Monitoring (Demo mit Zufallsagenten): KPI-Zeile, Weltkarten-Minimaps (grau = Wand, blaugrau = erkundeter Boden, grün = Startpunkt) und uPlot-Charts. Im Training zusätzlich: goldene Raute = gesichteter Exit, grüner Blitz bei Exit-Erfolg, Viridis-Heatmap.

Seed-Replay-Analyse (Swarm vs. PLR). Das Training wiederholt seit Etappe 2 bei 30 % der Episoden-Resets einen bereits *gelösten* Seed („Erfolgs-Swarm“). Ein Literaturabgleich stellt diese Semantik in Frage: Prioritized Level Replay [22] zeigt, dass das Replay von Levels mit *hohem Lernpotenzial*, also gerade *nicht* gemeisterten, Sample-Effizienz und Test-Generalisierung verbessert; das Wiederholen gemeisteter Levels ist der Fehlermodus,

den PLR behebt, und riskiert im Kontext des Projektziels (Generalisierung auf unbekannte Seeds) Layout-Memorierung. Da der 86 %-Referenzlauf jedoch *mit* Erfolgs-Swarm entstand, wird die Frage nicht per Meinung, sondern als A/B-Test entschieden (Erfolgs-Swarm vs. `--plr` vs. `--no-swarm`; Maßnahme B8 in `docs/BEWERTUNG_UND_PLAN.md`). Ein dabei entdeckter Designfehler wurde sofort behoben: Der Seed-Pool überlebte Phasenwechsel, obwohl ein „gelöster“ Seed nach dem Wechsel der Exit-Distanz eine andere Aufgabe beschreibt; der Pool wird jetzt bei jedem Phasenstart geleert. (*Der A/B-Test wurde noch am selben Tag im Zuge der Fehlersuche durchgeführt, siehe Abschnitt 10.6: Der Erfolgs-Swarm hatte auf Phase 1 keinen messbaren Effekt.*)

10.6 Etappe 6 (07.07.2026, nachmittags/abends): Systematische Fehlersuche vom Eval-Artefakt zur Batch-Größen-Korrektur

Die Stagnation der v11-Läufe wurde am Nachmittag des 07.07. in einer mehrstufigen, hypothesengetriebenen Fehlersuche aufgeklärt. Die folgende Darstellung enthält auch die verworfenen Hypothesen, da erst deren Ausschluss die jeweils nächste Maßnahme bestimmte. Zwei der Befunde, das Eval-Cap-Artefakt und die Wirkung der Batch-Größe, haben Tragweite über das konkrete Problem hinaus.

Schritt 1: Messartefakt entdeckt, Eval-Episoden-Caps. Ausgangspunkt war ein Widerspruch: Der Vergleichsrun Seed 1 stagnierte in Phase 1 bei 12–16 % (wie Seed 0), obwohl die Batch-Validierung derselben Konfiguration 52 % erreicht hatte. Eine Nachmessung des besten Seed-0-Checkpoints deckte die Ursache auf: Die am 06.07. zur Eval-Beschleunigung eingeführten Episoden-Caps (Phase 1: 600 statt 4000 Schritte) unterschätzen die Kompetenz erheblich, weil erfolgreiche LSTM-Episoden ohne Orakel lange Suchwege enthalten (Median 477, p95 $\approx$ 1 900, Maximum 3 239 Schritte). Dasselbe Modell misst sich bei Cap 600 als 48 %-Agent, bei Cap 4000 als 86 %-Agent (Abbildung 13). Damit war das 85 %-Phasenziel unter Cap 600 strukturell unerreichbar, und die Bestmodell-Selektion bevorzugte schnelle statt kompetente Politiken. Da die volle Messung nur $\approx$ 21 s kostet, wurden alle Caps auf 4000 (= Episoden-Limit der Umgebung = finales Eval-Protokoll) zurückgesetzt. *Lehre: Ein Eval-Protokoll-Detail (Cap) kann Gating, Modellselektion und Fortschrittswahrnehmung gleichzeitig verfälschen.*

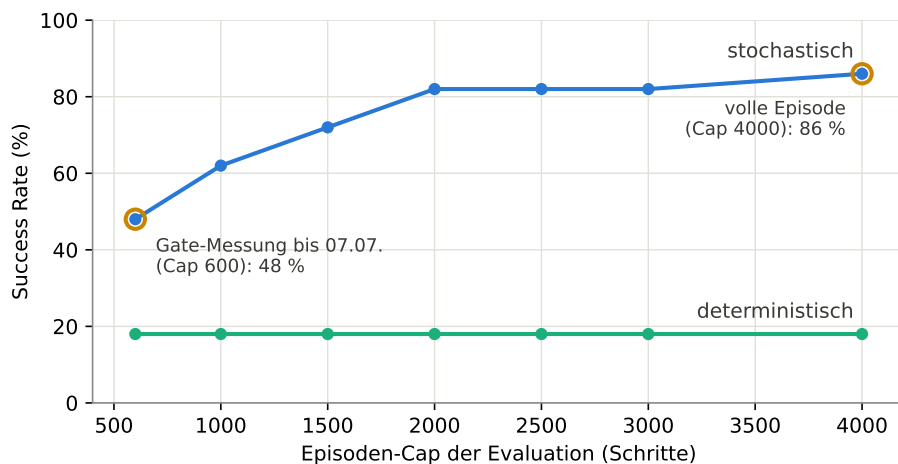


Abbildung 13: Erfolgsquote *desselben* Modells (bester Phase-1-Checkpoint, Seed 0) in Abhängigkeit des Episodenlimits der Evaluation, 50 Val-Seeds. Erzeugt mit `scripts/plot_etappe6_figures.py`.

Schritt 2: Neustart unter korrekter Messung, Stagnation ist real. Seed 1 wurde mit korrigierten Caps neu gestartet. Ergebnis: weiterhin Stagnation (16–28 % über 300 k

Schritte, `explained_variance` ≈ 0.1); der zweite stagnierende Seed in Folge. Damit war ein systematisches Problem belegt und eine Verdächtigenliste abzuarbeiten.

Schritt 3: Hypothesen nacheinander eliminiert (A/B-Läufe, Seed fixiert). Drei Komponenten des Trainings-Stacks wurden durch kontrollierte Vergleichsläufe mit identischem Seed geprüft:

- *Wand-/Loop-Penalties (Maßnahme B7)?* Entlastet ohne neuen Lauf: Die Batch-Validierung lief bereits auf der penalty-freien v11-Umgebung und lernte.
- *Erfolgs-Swarm (Maßnahme B8)?* Ein `--no-swarm`-Arm verlief deckungsgleich mit dem Kontrollarm; entlastet.
- *Streaming-Wrapper des Live-Monitorings?* Code-Inspektion (rein lesende Zugriffe) und ein Lauf ohne Wrapper: ebenfalls deckungsgleich; entlastet. (Nebenbefund behoben: Der Wrapper wurde bisher auch bei deaktivierter Live Map angelegt.)

Parallel dazu wurde die Batch-Validierung vom 06.07. *exakt reproduziert* (nackter RecurrentPPO ohne Curriculum-Stack, 150 k Schritte, `batch=64`): Sie erreichte am Endpunkt fast dieselben Werte wie damals (48 %/22 % vs. 52 %/22 %); der *Verlauf* dazwischen schwankte stark und unregelmäßig ($74 \rightarrow 44 \rightarrow 66 \rightarrow 24 \rightarrow 50$ %, Abbildung 14 oben). Zusätzlich wurde ausgeschlossen, dass sich Curriculum-Pfad und nackter Pfad überhaupt funktional unterscheiden: Gewichtsinitialisierung, Weltseed-Folgen und die ersten 8192 Trainingsschritte (Rewards, Aktionen, Values) sind zwischen beiden *bit-identisch*.

Schritt 4: Diagnose, hochvariable Lerndynamik statt defekter Code. Die Gesamtschau aller Läufe ergab: Mit `batch=64` ist die Lerndynamik hochvolatil (SR-Sprünge zwischen 10 und 74 % ohne Konvergenz), der Critic erreicht nur vereinzelte, nicht anhaltende `explained_variance`-Hochphasen. Die „Validierung“ der Batch-Größe am 06.07. beruhte auf einem günstigen Einzel-Messpunkt dieses Prozesses: einem Messwert aus einer stark schwankenden Kurve, nicht auf deren Verlauf. *Lehre: Hyperparameter-Entscheidungen brauchen Eval-Kurven, keine Endpunkte.*

Schritt 5: Hypothesentest Batch-Größe, bestätigt. Der einzige verbliebene Unterschied zum funktionierenden v2/v10-Rezept war `batch_size=64` statt 8 (bei `batch=8` erfolgen pro Rollout ≈ 8 -mal mehr Gradientenschritte auf denselben Daten; die v10-Reproduktion hatte damit EV bis 0.94 erreicht). Ein ansonsten identischer Lauf mit `batch=8` zeigte ein qualitativ anderes Bild (Abbildung 14): stetiger Anstieg auf **84–88 % stochastische SR** bei 125–150 k Schritten, die besten Phase-1-Werte des Projekts, und erstmals über **50 % deterministische SR** in Phase 1 (`batch=64`: nie über 26 %). Ab ≈ 100 k Schritten hält der Critic anhaltende EV-Hochphasen (26 % der Iterationen > 0.5 , gegenüber 3 % bei `batch=64`). Konsequenz: `batch_size` wurde auf 8 zurückgesetzt; der Geschwindigkeitsvorteil von `batch=64` (Faktor 2,1) war mit instabilem Lernen erkaufte und damit wertlos.

Ergebnis der Etappe. Die v11-Stagnation hatte zwei unabhängige, sich überlagernde Ursachen: ein *Messartefakt* (Eval-Caps) und eine *echte Regression* (`batch=64`). Trainings-Stack, Seed-Replay, Wrapper und Umgebungsrevision wurden dabei experimentell entlastet, ein für die weitere Arbeit wertvolles Nebenergebnis, da die v11-Umgebung damit als methodisch sauber bestätigt ist. Die Konfiguration für die finalen Mehrfach-Läufe steht damit fest (v11-Umgebung, `batch=8`, Cap-4000-Evals, kurvenbasierte Bewertung).

10.7 Umgebungsrevision v11 (06.07.2026)

Eine systematische Überprüfung der Umgebung gegen Best Practices des Environment- und Reward-Designs führte zu zehn dokumentierten Änderungen. Die folgenden sieben betreffen Umgebung, Reward und Evaluation; die übrigen drei (Device- und Batch-Benchmark, Konsistenz- und Usability-Bereinigung) sind in Abschnitt 10.8 beschrieben.

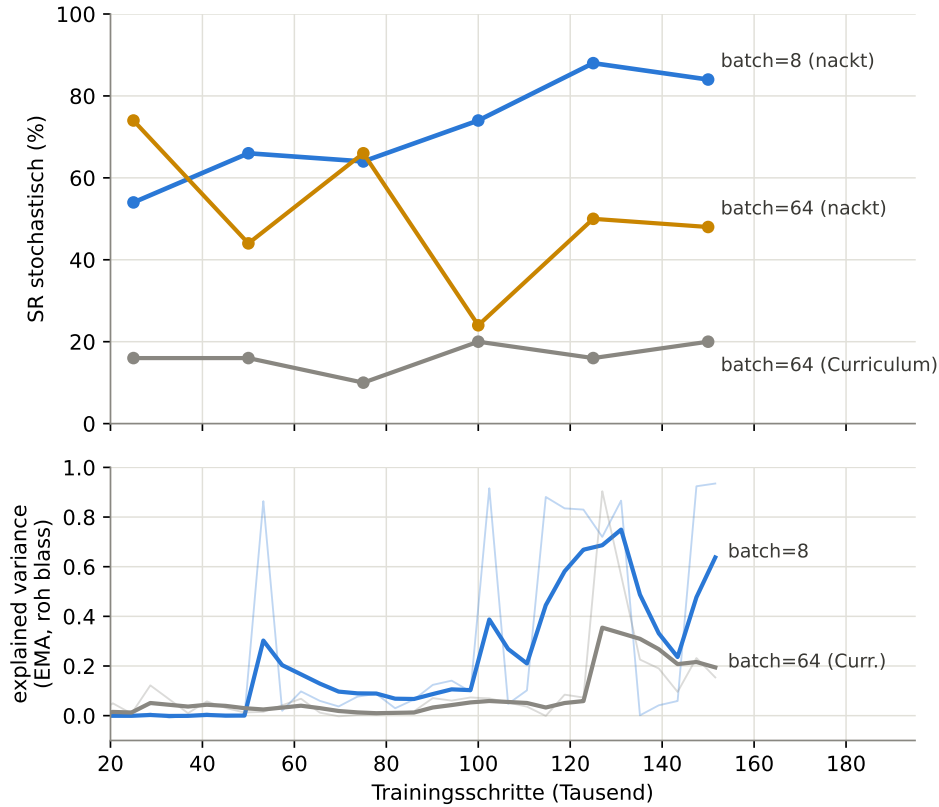


Abbildung 14: Batch-Größen-Forensik (alle Läufe Seed 1, Phase-1-Setup, 50 Val-Seeds, Limit 4000). Oben: stochastische Erfolgsquote. Unten: `explained_variance` des Critics, EMA-geglättet mit blassen Rohkurven. Erzeugt mit `scripts/plot_etappe6_figures.py`.

1. **Tote Features entfernt:** Energie und Inventar waren konstant; Observation 231 → 229 Dimensionen (Tabelle 2).
2. **Straf-Stacking entschärft:** Wand-Penalty entfernt, Loop-Penalty $-0.15 \rightarrow -0.05$ (Tabelle 3).
3. **Exit-Platzierung nach BFS-Laufweg:** Zuvor wurden Kandidaten nach *Luftlinie* gewählt; der reale Laufweg bei nominell „35–45“ streute auf 42–75 Tiles ($\varnothing$ 55). Nach dem Fix liegen alle 150 Eval-Seeds exakt im Sollbereich (Abbildung 15); das Curriculum steuert die Schwierigkeit damit erstmals präzise.
4. **Prozess-globales Config-Leck behoben:** Die Weltgenerierungs-Konfiguration ist im C++-Kern global; parallel existierende Umgebungen überschrieben sich gegenseitig (u. a. verschob der Eval-Callback die Phase-3-Trainingsverteilung unbemerkt). Jede Umgebung stempelt ihre Konfiguration nun bei jedem Reset neu.
5. **Mining/Bauen/Kampf aus dem RL-Pfad entfernt** (Binding erlaubt nur noch Aktionen 0–3; tote Reward-Terme gestrichen).
6. **Entropie-Annealing korrigiert:** stetig $0.05 \rightarrow 0.001$ statt Sprung $0.05 \rightarrow 0.01$ beim Phasenwechsel.
7. **Eval-Callback:** phasengerechtes Gating (Tabelle 4), Episoden-Caps pro Phase, Det+Stoch-Doppelmessung.

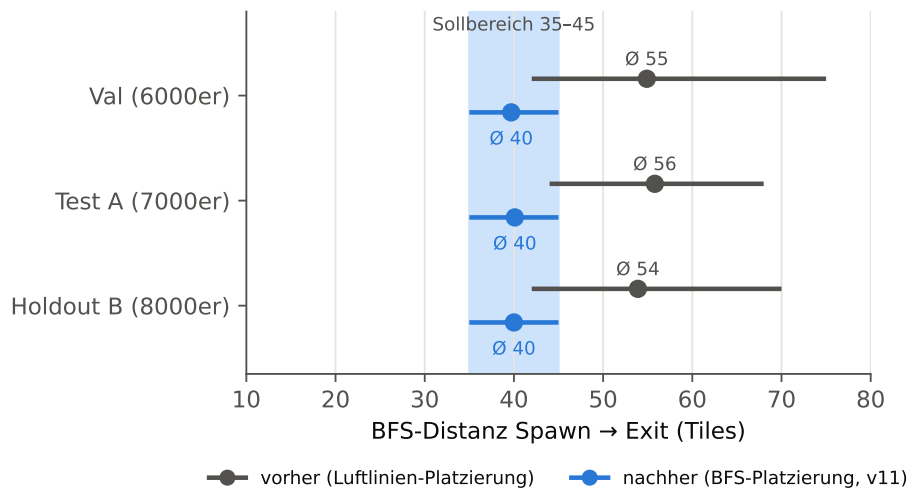


Abbildung 15: Wirkung der BFS-basierten Exit-Platzierung: tatsächliche Laufweg-Distanzen (min/Ø/max über je 50 Seeds) vor und nach dem Fix.

Alle Änderungen wurden einzeln verifiziert (Lösbarkeit 150/150, Oracle-Agent 10/10 mit exakt BFS-Distanz Schritten, Obs-Layout-Konsistenz, Ablehnung unzulässiger Aktionen). Da sich die Aufgabenverteilung dadurch ändert, sind Ergebnisse ab v11 als neue Baseline zu behandeln.

10.8 Performance-Analyse: Training beschleunigen

10.8.1 CPU schlägt Apple-GPU (MPS)

Sämtliche Läufe dieser Arbeit entstanden auf einem Notebook der Apple-M-Serie (M1 Pro, 8 Performance-Kerne, macOS arm64); als Rechen-Device stand damit neben der CPU die integrierte GPU über das MPS-Backend zur Verfügung. Ein kontrollierter Benchmark (identisches Trainings-Setup, 8192 Steps) zeigt: Die GPU ist für dieses kleine Netz (LSTM 256, 229 Inputs) in *jeder* Konfiguration langsamer als die CPU; der Kernel-Dispatch-Overhead pro Mini-Update übersteigt den Rechengewinn (Abbildung 16). Dies deckt sich mit der Empfehlung der SB3-Dokumentation, PPO ohne CNN-Politik primär auf der CPU zu betreiben [30].

10.8.2 Batch-Größe: vermeintliche Validierung (06.07.), am Folgetag widerlegt

Mit `batch_size=8` fallen pro Rollout 5 120 Gradient-Updates an, mehr Optimizer-Schritte als Umgebungsschritte. Da die Root-Cause-Analyse (Tabelle 14) die Batch-Größe nie isoliert getestet hatte, wurde ein Validierungslauf durchgeführt: 150k Phase-1-Steps mit `batch_size=64` (Abbildung 17). Ergebnis: gesunde Lernmetriken (`approx_kl` 0.01–0.06, `explained_variance` bis 0.72) und **52 % stochastische / 22 % deterministische SR** nach nur 150k Steps, bei **187 statt 88 FPS**. `batch_size=64` wurde daraufhin übernommen.

Diese Schlussfolgerung hielt der Überprüfung nicht stand. Die Forensik vom 07.07. (Abschnitt 10.6) zeigte per exakter Reproduktion, dass der Validierungslauf ein günstiger Einzel-Schnappschuss einer stark schwankenden Lerndynamik war: Der Endpunkt (52 %) war reproduzierbar, der Verlauf dorthin sprang jedoch regellos zwischen 24 und 74 %, und in vollen Curriculum-Läufen stagnierte `batch=64` bei 16–28 %. Mit `batch_size=8` lernt dieselbe Konfiguration stetig auf 84–88 % (Abbildung 14). Die Batch-Größe *war* also doch ein kritischer v2-Erfolgsfaktor; sie wurde auf 8 zurückgesetzt. Die Lehre daraus: *Einzel-Messpunkte volatiler Prozesse validieren nichts.*

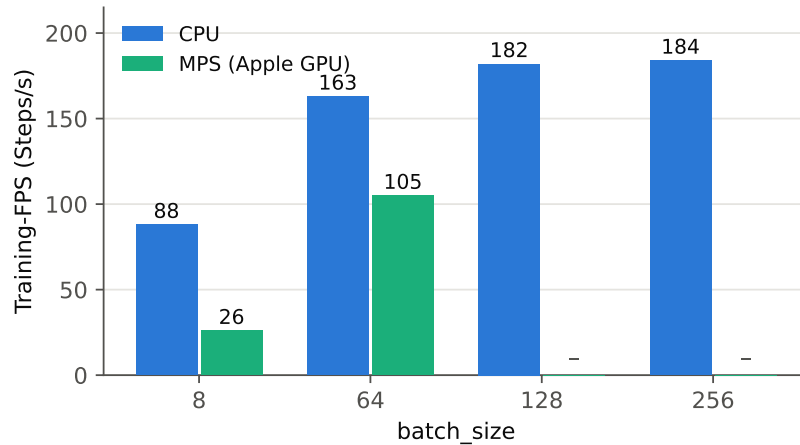


Abbildung 16: Training-FPS nach Device und Batch-Größe (RecurrentPPO, 16 Umgebungen, Apple M-Serie). MPS wurde für $\text{batch} \geq 128$ nicht weiter gemessen, da bereits bei 64 unterlegen.

10.9 Zeitleiste und Lessons Learned

Tabelle 15 fasst den Verlauf der vorangegangenen Abschnitte chronologisch zusammen; die anschließende Liste hält die übertragbaren Erkenntnisse fest.

Lessons Learned (negative Ergebnisse mit Erkenntniswert):

- *Visited-Mask in der Observation* destabilisiert den Argmax (jede 1-Bit-Änderung kann die Richtungspräferenz kippen).
- *Naiver Curriculum-Transfer* zerstört die Vorgänger-Politik; Mixed-Distribution bzw. leistungsbasierte Phasenwechsel sind nötig.
- *Feature-Kodierung schlägt Feature-Auswahl*: Der BFS-Gradient war nie „zu schwach“, sondern falsch kodiert (Delta-Fix: Signal $32\times$ stärker).
- *Straf-Stacking* (Wand-, Loop-, Stagnations-Penalties) macht Stillstand lokal optimal; Information gehört in die Observation, nicht in den Reward.
- *Hyperparameter dominieren Features*: Alle v7-Feature-Ideen scheiterten an verstellten Basis-Hyperparametern; `ent_coef` war der entscheidende Faktor. Ohne das Changelog wäre die Root-Cause-Analyse (v2-Reverse-Engineering) nicht möglich gewesen.
- *Eval-Protokoll ist Teil des Experiments*: Data-Leakage über den Callback und das Gating auf einer signal-losen Metrik (det. SR bei hoher Entropie) verzerrten Selektion und Laufzeit unbemerkt.
- *Eval-Episoden-Caps verfälschen die Messung*: Dasselbe Modell misst sich bei Cap 600 als 48-%-, bei Cap 4000 als 86-%-Agent; eine als harmlos gedachte Beschleunigung machte das Phasenziel unerreichbar (Abschnitt 10.6).
- *Einzel-Snapshots sind keine Validierung*: Die batch=64-Freigabe beruhte auf einem Messpunkt einer stark schwankenden Kurve; die exakte Reproduktion traf den Endpunkt, zeigte aber, dass der Verlauf dazwischen volatil war. Hyperparameter-Entscheidungen nur auf ganzen Eval-Kurven.

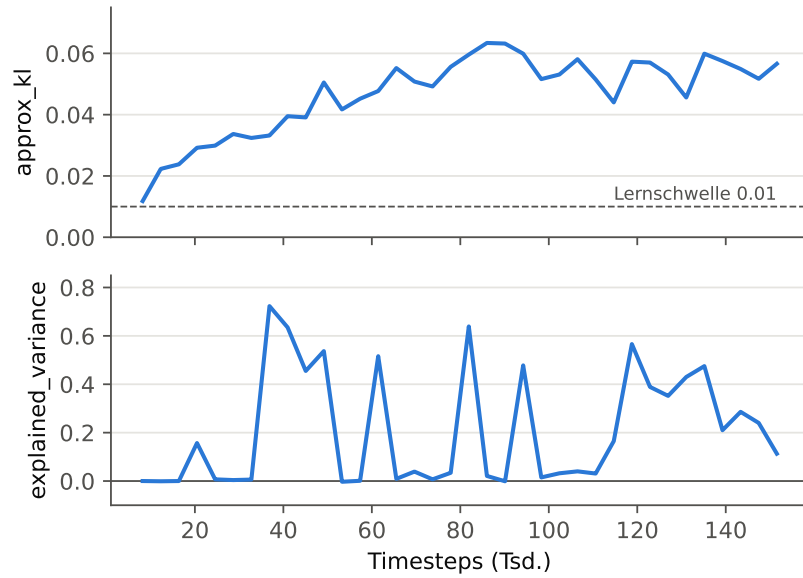


Abbildung 17: Validierungslauf `batch_size=64` (Phase-1-Setup, Env v11), Stand 06.07.: `approx_kl` und `explained_variance` über der Zeit. Zur späteren Widerlegung dieses Befunds siehe Abschnitt [10.6](#).

- *Negative Kontrollen lohnen sich:* Der bit-genaue Vergleich von Curriculum-Pfad und nacktem Trainingspfad schloss eine ganze Verdächtigen-Klasse (Stack-Bugs) in Minuten aus und lenkte die Suche auf die Hyperparameter.

Datum	Meilenstein	Ergebnis (Testset A)
12.05.	Spiel-Engine & Weltgenerierung spielbar	–
16.05.	DQN-Fehlstart → PPO; BFS-Kraftfeld-Bug gefixt	Oracle 0/3 → 3/3
16.05.	Curriculum P1; naiver Transfer kollabiert → Mixed-Dist.	56 % → 86 % stoch
16.05.	Phase 4 (Reward-Tuning), 3-Läufe-Protokoll	98 % stoch / 2 % det
18.05.	Delta-BFS-Kodierung + Temperature-Sampling	100 % stoch / 42 % det
07./08.06.	RecurrentPPO ohne BFS-Orakel (Kernbeitrag) ; Lauf gestartet 07.06., ausgewertet 08.06.	86 % / 36 %
08.06.	POMDP-These; Callback-Bugs; Swarm/Live-Map	–
11.06.	Data-Leakage behoben (Val-Set 6000er); Lösbarkeit 100 %	–
13.–15.06.	v6–v9 scheitern → Root-Cause (ent_coef/batch/n_steps)	0–12 %
25.06.	v10-Reproduktion abgeschlossen (Referenzmodell)	86 % (Val)
06.07.	Env-Revision v11 (10 Änderungen); batch=64 „validiert“ (am 07.07. widerlegt)	52 % stoch @150k (P1)
07.07.	v11-Läufe Seed 0/1 stagnieren; Monitoring-Redesign; Swarm-Analyse (B8) + Pool-Fix	16–28 % stoch (P1, Val)
07.07.	Forensik (Etappe 6) : Eval-Cap-Artefakt gefixt; Swarm/Wrapper/Stack entlastet; batch 64 → 8	84–88 % stoch / 52 % det (P1, Val)
08.07.	v12-Läufe Seeds 1–3 (batch=8) ; Gap-Experimente E1–E3 + Temperatur-Sweep	73,3 % / 80,0 % (A/B, stoch, $n = 3$)
17.07.	Aufstockung auf $n = 7$ (Seeds 4–7); Alternativen ausgeschlossen	65,7 % / 66,9 % (A/B, stoch, $n = 7$)

Tabelle 15: Zeitleiste der RL-Entwicklung (Auswahl; vollständig in `CHANGELOG.md`). SR-Angaben je nach Etappe mit unterschiedlichem Protokoll; Werte vor dem 11.06. ohne strikte Val/Test-Trennung.

Literatur

- [1] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020. <https://arxiv.org/abs/1912.01588>.
- [2] Sebastian Risi and Julian Togelius. Increasing generality in machine learning through procedural content generation. *Nature Machine Intelligence*, 2(8):428–436, 2020. <https://www.nature.com/articles/s42256-020-0208-5>.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018. <http://incompleteideas.net/book/the-book-2nd.html>.
- [4] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- [5] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. *arXiv preprint arXiv:1507.06527*, 2015. <https://arxiv.org/abs/1507.06527>.
- [6] Satinder P Singh, Tommi Jaakkola, and Michael I Jordan. Learning without state-estimation in partially observable markovian decision processes. In *Proceedings of the Eleventh International Conference on Machine Learning (ICML)*, pages 284–292, 1994. <https://www.cs.utexas.edu/~shivaram/readings/b2hd-SinghJJ1994.html>.
- [7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. <https://www.nature.com/articles/nature14236>.
- [8] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. <https://arxiv.org/abs/1506.02438>.
- [9] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1928–1937. PMLR, 2016. <https://arxiv.org/abs/1602.01783>.
- [10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>.
- [11] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <http://jmlr.org/papers/v22/20-1364.html>.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [13] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

- [14] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009. <https://dl.acm.org/doi/10.1145/1553374.1553380>.
- [15] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, et al. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *Advances in Neural Information Processing Systems*, 36, 2023. <https://arxiv.org/abs/2306.13831>.
- [16] Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021. <https://arxiv.org/abs/2109.06780>.
- [17] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011. <https://ieeexplore.ieee.org/document/5963131>.
- [18] Steven Kapturowski, Georg Ostrovski, John Quan, Rémi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- [19] Marco Pleines, Matthias Pallasch, Frank Zimmer, and Mike Preuss. Generalization, mayhems and limits in recurrent proximal policy optimization. *arXiv preprint arXiv:2205.11104*, 2022. <https://arxiv.org/abs/2205.11104>.
- [20] Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. POPGym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023. <https://arxiv.org/abs/2303.01859>.
- [21] Jake Grigsby, Linxi Fan, and Yuke Zhu. AMAGO: Scalable in-context reinforcement learning for adaptive agents. In *International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2310.09971>.
- [22] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning (ICML)*, pages 4940–4950. PMLR, 2021. <https://arxiv.org/abs/2010.03934>.
- [23] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017. <https://arxiv.org/abs/1705.05363>.
- [24] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019. Preprint 2018, <https://arxiv.org/abs/1810.12894>.
- [25] Dibya Ghosh, Jad Rahme, Aviral Kumar, Amy Zhang, Ryan P Adams, and Sergey Levine. Why generalization in RL is difficult: Epistemic POMDPs and implicit partial observability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. <https://arxiv.org/abs/2107.06277>.
- [26] Norbert L. Kerr. HARKing: Hypothesizing after the results are known. *Personality and Social Psychology Review*, 2(3):196–217, 1998. https://journals.sagepub.com/doi/10.1207/s15327957pspr0203_4.

- [27] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. <https://arxiv.org/abs/2003.12206>.
- [28] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. <https://arxiv.org/abs/1709.06560>.
- [29] Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning. In *International Conference on Machine Learning (ICML)*, pages 3878–3887. PMLR, 2018. <https://arxiv.org/abs/1806.05635>.
- [30] Stable-Baselines3 Contributors. Stable-baselines3 documentation: PPO, 2025. <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>, abgerufen am 25.07.2026: „PPO is meant to be run primarily on the CPU, especially when you are not using a CNN“.